

# Parallel Generation of Abelian Pattern-Free Words on Three Letters

Aleksandr Gavrilenko

---

## Abstract

An abelian square is a word that can be divided into two consecutive parts such that the first part is a permutation (anagram) of the second part, that is, the number of occurrences of each letter is the same for the both parts. This thesis explores the programmatic approach to generation of long words not containing almost (that is, shortest one letter repetitions are allowed) abelian squares using Rust programming language.

The thesis briefly covers the history of the related field of combinatorics on words, the benefits of using Rust and the process of implementation of two programs that can be used to generate the longest word that does not contain almost abelian squares.

As the main result of the thesis, two programs utilizing different approaches were created. The programs can be used for the later research in the field.

*Key words:* Rust, Abelian Square Avoidance, Parallel Computing

---

## Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                | <b>4</b>  |
| 1.1      | Preliminaries . . . . .                            | 4         |
| 1.2      | History . . . . .                                  | 4         |
| 1.3      | Current State . . . . .                            | 5         |
| <b>2</b> | <b>Language Choice Justification</b>               | <b>6</b>  |
| <b>3</b> | <b>Introduction to Rust Syntax</b>                 | <b>7</b>  |
| 3.1      | Primitive Types and Variable Declaration . . . . . | 7         |
| 3.2      | Memory Management . . . . .                        | 9         |
| 3.3      | Custom Data Types . . . . .                        | 11        |
| 3.4      | Control Flow . . . . .                             | 13        |
| 3.5      | Polymorphism . . . . .                             | 15        |
| <b>4</b> | <b>Implementation</b>                              | <b>17</b> |
| 4.1      | Task Overview . . . . .                            | 17        |
| 4.2      | Single-Threaded Code . . . . .                     | 18        |
| 4.3      | Parallelization . . . . .                          | 23        |
| 4.3.1    | Thread Communication . . . . .                     | 23        |
| 4.3.2    | Generator Structure . . . . .                      | 26        |
| <b>5</b> | <b>Discussion</b>                                  | <b>28</b> |

## List of Figures

|   |                                                                       |    |
|---|-----------------------------------------------------------------------|----|
| 1 | The Computer Language Benchmarks Game: Fastest Languages. Source:[11] | 6  |
| 2 | Comparison of Performance of Thread Communications Methods . . .      | 26 |

## List of Tables

|   |                                   |   |
|---|-----------------------------------|---|
| 1 | Primitive Types in Rust . . . . . | 8 |
|---|-----------------------------------|---|

## List of Listings

|    |                                                              |    |
|----|--------------------------------------------------------------|----|
| 1  | Variable Declaration Demonstration . . . . .                 | 9  |
| 2  | Memory Management Demonstration . . . . .                    | 11 |
| 3  | Complex Data Types Demonstration . . . . .                   | 13 |
| 4  | Control Flow Demonstration . . . . .                         | 15 |
| 5  | Polymorphism Demonstration . . . . .                         | 17 |
| 6  | AA2F Full Check . . . . .                                    | 19 |
| 7  | Single-Threaded Extender . . . . .                           | 20 |
| 8  | Optimized AA2F Check . . . . .                               | 21 |
| 9  | Using Closure Function to Halt the Program . . . . .         | 21 |
| 10 | Functions Checking if Word is Template Vector Free . . . . . | 22 |
| 11 | Testing Tread Communication Methods . . . . .                | 25 |
| 12 | Generator Definition and Implementation . . . . .            | 28 |

## 1. Introduction

### 1.1. Preliminaries

The reader might ignore this subsection if they are familiar with terminology.

An *alphabet*  $\Sigma$  is a finite non-empty set of elements called *letters*. A *word* over  $\Sigma$  is a sequence of letters belonging to the  $\Sigma$ . The *empty word* is represented by  $\lambda$ . The set of all finite words [all non-empty finite words] is denoted as  $\Sigma^*$  [ $\Sigma^+$ ]. On the  $\Sigma^*$  the binary operation of *catenation* is defined (with the neutral element being  $\lambda$ ). The structures  $\Sigma^*$  and  $\Sigma^+$  are called the *free monoid* and the *free semigroup* respectively.

Let  $w = x_1 \cdots x_m, x_i \in \Sigma$ . The *length* of the word  $w$ , denoted by  $|w|$ , is the number of occurrences of letters in the word, i.e.,  $|w| = m$ . Let  $\Sigma = \{a_1, \dots, a_n\}$ . The number of occurrences of the letter  $x, x \in \Sigma$ , in  $w$  is denoted by  $|w|_x$ , or by  $|w|_i$  if  $x = a_i$ . The notation  $\psi(w)$  stands for *Parikh vector* of  $w$ ;  $\psi(w) = (|w|_1, \dots, |w|_n)$ .

A word  $u$  is called a *factor* of word  $w$  if there exist words  $p$  and  $s$  such that  $w = pus$ . If  $p = \lambda$  [ $s = \lambda$ ], then  $u$  is called a *prefix* [*suffix*] of  $w$ .  $\text{FACT}(w)$ ,  $\text{PREFIX}(w)$  and  $\text{SUFFIX}(w)$  denote sets of all factors, prefixes and suffixes of  $w$  respectively.

For a given integer  $k, k \geq 2$ , a *k-repetition* is a non-empty word of the form  $R^k$ . An *abelian k-repetition* is a non-empty word of the form  $P_1 \cdots P_k$ , where  $\psi(P_i) = \psi(P_j)$  for all  $1 \leq i, j \leq k$ . [Abelian] 2-repetitions and 3-repetitions are also called [abelian] squares and cubes. A word is called [abelian] *k-repetition free* if it does not contain a factor that is an [abelian] k-repetition. If it is possible to construct infinite [abelian] k-repetition free words over a given alphabet  $\Sigma$ , then [abelian] k-repetitions are said to be *avoidable* over  $\Sigma$ .

A *morphism*  $h$  is a mapping between free monoids  $\Sigma^*$  and  $\Delta^*$  satisfying  $h(uv) = h(u)h(v)$  for every  $u$  and  $v$  in  $\Sigma^*$ . In particular,  $h(\lambda) = \lambda$ . An *endomorphism* (usually denoted by  $g$ ) is a morphism with  $\Sigma = \Delta$ .

For a given integer  $k \geq 2$  a morphism  $\sigma : \Sigma^* \rightarrow \Delta^*$  is called *k-free* [*a-k-free*], if all the words  $\sigma(w)$  are k-free [a-k-free] for every k-free [a-k-free] word  $w \in \Sigma^*$ .

### 1.2. History

At the beginning of 20th century Axel Thue laid the foundation of the combinatorics on words in his remarkable paper 'Über unendliche Zeichenreihen' [1]. However, the

paper remained largely unnoticed at the time. The lack of interest in the area, more or less, continued until 1961 when Erdős [2] raised a question whether abelian squares can be avoided in words of infinite length over a four-letter alphabet.

In 1970, Pleasants [3] proved existence of an infinite abelian square-free word over five letters. Four years later Etringer et al. [4] demonstrated that every infinite word over two letters contains arbitrarily long abelian squares. In 1979, Dekking [5] proved existence of (1) an infinitely long abelian 4-repetition free sequence over two letters and (2) an infinitely long abelian cube-free sequence over three letters.

The next notable achievement happened in 1992 when Keränen [6] published endomorphism  $g_{85}$  which allowed to generate abelian square-free words of unlimited length over 4 letter-alphabet. In 1993 Carpi [7] formulated sufficient conditions for morphisms to be  $a$ - $k$ -free and confirmed  $g_{85}$  to be abelian square free. Additional mappings, based on  $g_{85}$  were discovered by Carpi [8] in 1998. As a continuation of research,  $12^4$  morphisms was presented by Keränen [9] in 2009.

### *1.3. Current State*

The next step after discovering a way to avoid abelian squares over 4 letters would be to do so over 3 letters. However, no such words having length greater than 7 exist. Therefore, nowadays the research of pattern avoidance in words is more focused on almost abelian square-free words ( $a$ - $a$ -2-free or AA2F) over a 3 letter alphabet, i.e., words that allow repetitions  $xx$  and  $xxx$  for a letter  $x$ , but do not contain other abelian squares.

This project takes a programmatic approach to the problem, attempting to generate a significant amount of relevant data for further research.

The first major task is to create a program which will attempt to generate  $a$ - $a$ -2-f words in order of increasing length, using various search optimization techniques. The second task is to create a program that will read a list of  $a$ - $a$ -2-f words from input file, and sort the words into two groups. The first group should include words that can be extended both to the left and to the right for predetermined length, and, consequently, have a probability of being used as a building block for longer words. The second group should include words that cannot be extended.

## 2. Language Choice Justification

The task described in the 1.3 is computation heavy due to exponential growth (as the worst-case scenario) of the number of words required to be checked as the length increases. Because of this, the most prioritized features of languages are (1) high performance and (2) simple parallelization. Effectively, the first requirement dismisses interpreted languages. Garbage collected languages are also undesirable, because GC algorithms need to consume the computational resources themselves.

Finally, the author made decision to use Rust [10] for the following reasons:

1. Performance. Rust demonstrates speed comparable to that of C and C++ languages, as demonstrated by benchmarks [11, 12]. To the large degree this can be attributed to the fact that rustc (the compiler of Rust) is a fronted for LLVM, which is most well-known for being the backend for the Clang —the C-family language compiler used by Apple.

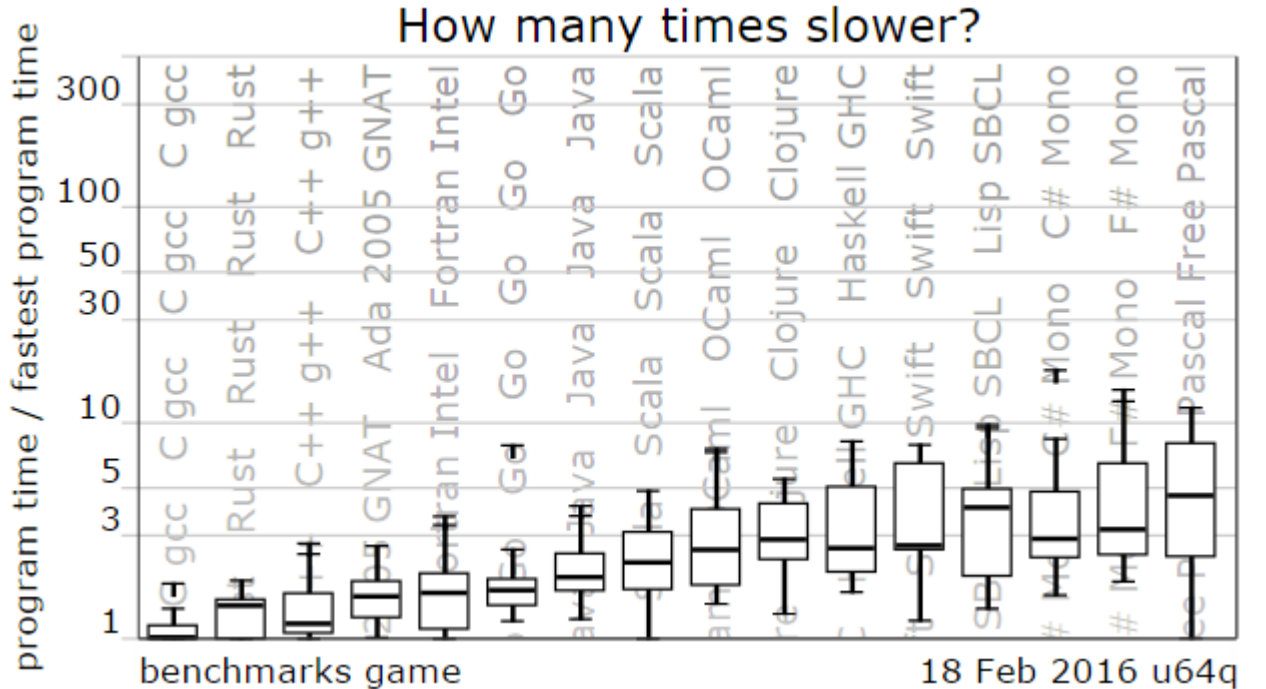


Figure 1: The Computer Language Benchmarks Game: Fastest Languages. Source:[11]

2. Parallelization. Using safe subset of Rust (henceforth written as *Safe Rust* when opposed to *Unsafe Rust*, and simply *Rust* otherwise), programmer is guaranteed to avoid data races [13, 14]. Race conditions are still possible, but guaranteed

not to cause memory violation in Safe Rust [13]. It should be noted, that while data races have a rigidly defined conditions to be considered such, race conditions lean closer to being a logical error, and therefore it is sufficiently harder (if at all possible) to prevent them programmatically [15].

3. Memory management. By utilizing novel approach to memory management (only used before in an experimental C dialect —Cyclone), Rust guarantees memory safety. That is, the language prevents dereferencing null or dangling pointers, reading uninitialized memory and violating the strictly defined (see subsection 3.2) pointer aliasing rules [13].

### 3. Introduction to Rust Syntax

Rust is positioned as a low-level systems languages, potentially aiming to replace C. Due to this, the syntax is largely reminiscent of that of C in order to ease the transition from C to Rust for experienced programmers. Nevertheless, Rust brings concepts from functional programming (henceforth *FP*) languages [16], which might be unfamiliar for a hypothetical C programmer. This section aims to cover some of the differences and familiarize the reader with the Rust syntax and the language idioms. To accomplish this, code listings will frequently be employed. Some material will be presented as commentaries in the listings. The reader should be aware that this is not an exhaustive description of the language; the references [17–19] are advised for deepening the understanding.

#### 3.1. Primitive Types and Variable Declaration

Table 1 describes the majority of the primitive types that exist in Rust, as well as **Box**, **Vec** and **String** types, which are not primitive, but widely used.

Table 1: Primitive Types in Rust

| Type                | Size                                                             | Meaning                                                                                                                                                                                           |
|---------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| bool                | 1 byte                                                           | A boolean value: either true or false                                                                                                                                                             |
| char                | 4 bytes                                                          | A Unicode character                                                                                                                                                                               |
| i8                  | 1 byte                                                           | A signed integer number                                                                                                                                                                           |
| i16                 | 2 bytes                                                          |                                                                                                                                                                                                   |
| i32                 | 4 bytes                                                          |                                                                                                                                                                                                   |
| i64                 | 8 bytes                                                          |                                                                                                                                                                                                   |
| u8                  | 1 byte                                                           | An unsigned integer number                                                                                                                                                                        |
| u16                 | 2 bytes                                                          |                                                                                                                                                                                                   |
| u32                 | 4 bytes                                                          |                                                                                                                                                                                                   |
| u64                 | 8 bytes                                                          |                                                                                                                                                                                                   |
| isize               | Varies: 4 bytes for x86 programs and 8 bytes for x86_64 programs | A signed integer number size of which depends on size of pointer of the underlying machine                                                                                                        |
| usize               |                                                                  | An unsigned integer number size of which depends on size of pointer of the underlying machine                                                                                                     |
| f32                 | 4 bytes                                                          | A floating point number                                                                                                                                                                           |
| f64                 | 8 bytes                                                          |                                                                                                                                                                                                   |
| Box<T>              | <i>size(usize)</i>                                               | A unique pointer to heap-allocated data of type T.                                                                                                                                                |
| [T; N]              | $N * \text{size}(T)$ bytes                                       | An array of values of type T. Values can be accessed by index as <code>myArray[n]</code> . Arrays use zero-based numeration in Rust. Arrays can automatically coerce to slices                    |
| &[T]                | $2 * \text{size}(usize)$                                         | A slice: a dynamically-sized view into a contiguous sequence                                                                                                                                      |
| Vec<T>              | $3 * \text{size}(usize)$                                         | Is a pointer to a heap-allocated growable list of values of type T. Can automatically coerce to slices                                                                                            |
| (type1, ..., typeN) | $\text{size}(\text{type1}) + \dots + \text{size}(\text{typeN})$  | A tuple: ordered list of values of different types. Values can be accessed by index as <code>myTuple.0</code> . Index should be defined at compile-time. Tuples use zero-based numeration in Rust |
| str                 | unsized                                                          | A stack-allocated sequence of Unicode characters. Because it is unsized, there is no way to pass it by value                                                                                      |
| String              | $3 * \text{size}(usize)$                                         | Is a pointer to a heap-allocated sequence of Unicode characters                                                                                                                                   |

Variables in Rust are immutable by default. To declare a variable as mutable, programmer needs to explicitly specify mutability with the **mut** keyword. Aliasing is strictly limited in Rust, the exact rules are covered in subsection 3.2. Listing 1 demonstrates mutable and immutable variable declaration.

```

1 fn main() {
2     //Use the let keyword to declare variables
3     //Variables are immutable by default

```

```

4      let day: i64 = 15;
5      //Mutable variables are declared via let mut
6      //Types can be inferred by compiler if there is enough data
7      //Integer literals can have suffixes, e.g. u8, i8, ..., u64, i64
8      //Floating point literals have suffixes f32 and f64
9      let mut year = 2010i32;
10     year += 5;
11
12     //Immutable variables can be initialized with a delay
13     let version:f32;
14     version = 1.0f32;
15
16     //Type 'str' is unsized, and as such cannot be stored directly.
17     //The reference to it is used.
18     let month: &str = "May";
19
20     //'String's', however, can be stored in a variable directly.
21     let lang: String = "Rust".to_owned();
22
23     println!("The version {:.1} of {} was announced on {} {}, {}.",
24             version, lang, month, day, year);
25     //Output: The version 1.0 of Rust was announced on May 15, 2015.
26
27     let my_array: [i64; 3] = [0; 3];
28     //or let my_array: [i64; 3] = [0, 0, 0];
29     let my_vector: Vec<i64> = vec![1, 2, 3, 4, 5];
30
31     //tuple declaration
32     let is_aa2f_pair: (String, bool) = ("abacaba".to_owned(), true);
33
34     //Tuples can be destructured into variables
35     let (word, is_aa2f) = is_aa2f_pair;
36     println!("Word {}'s aa2f status is: {}", word, is_aa2f);
37     //Output: Word abacaba's aa2f status is: true
38
39 }

```

Listing 1: Variable Declaration Demonstration

### 3.2. Memory Management

Rust uses memory management model first implemented in the Cyclone language [20, 21]. At any given point of time, each unit of data is owned by exactly one binding. When this binding goes out of scope, the data is deallocated. This concept applies to heap-allocated data as well. When, for example, a smart unique pointer (**Box**) goes out of scope, the heap-allocated data it was pointing to is deallocated. Ownership can be transferred (or the data can be moved) to another binding. This approach allows to deterministically free memory without enforcing a programmer to do it manually (which is error-prone) or using a garbage collector (which is slow).

Data can also be borrowed, both mutably or immutably. A borrow cannot outlive the owned binding. This is guaranteed by specifying lifetimes, however, more often than not, the compiler can elide them automatically. There can be only one mutable borrow (and no immutable ones) or indefinitely many immutable borrows. When the borrow goes out of scope, the borrowed data is returned. If the need arises to truly share the data (instead of borrowing), it can be done by using a reference-counted pointer

(**Rc**<**T**>) or its thread-safe atomic version (**Arc**<**T**>). When doing so, one still needs to synchronize the access to the shared data, which often leads to using types similar to **Arc**<**Mutex**<**T**>>.

```

1  fn consume_vec(arg_vec: Vec<i64>) {
2  }
3
4  fn demonstrate_borrowing_and_deallocation() {
5      let mut x = 0i64;
6      let y = 10i64;
7      //Boxing the data allocates it on the heap
8      //and creates a unique pointer to it
9      let mut z = Box::new(20i64);
10     //Blocks are denoted by curly braces.
11     //At the end of each block, all owned variables are deallocated
12     //and all borrows are returned.
13     {
14         //Mutable variables can be borrowed either
15         //by mutable reference or by immutable reference
16
17         //x_1 is, in terms of C, a constant pointer to
18         ↪ non-constant data
19         let x_1: &mut i64 = &mut x;
20         %//There can be only one mutable reference:
21         %//let mut x_2: &mut i64 = &mut x; //Compilation error:
22         %//???
23
24         %//Immutable data cannot be mutably borrowed
25         %//!!!!!!!!!!!!!!
26         //Alternative syntax, note the "ref" keyword
27         //and lack of "&" on the right-hand side
28         let ref y_1 = y;
29         //The number of immutable references is not limited
30         let ref y_2 = y;
31
32
33         //Note that the z_1 is immutable, but
34         //the reference to boxed data it holds is mutable
35         let z_1: &mut Box<i64> = &mut z;
36
37         //Immutable variables can be only borrowed immutably
38         //let mut y_1: &mut i64 = &mut y; //Compilation error:
39         //Cannot borrow immutable variable as mutable
40
41         let y_1: &i64 = &y;
42
43         //References need to be dereferenced by "*"
44         *x_1 = *x_1 + 5;
45         //Note the double dereferencing for "z_1"
46         //You need to dereference the reference to access the Box
47         //And you need to dereference the Box to access the data
48         **z_1 += 5;
49
50         // *y_1 += 5; //Compilation error:
51         //Cannot assign to immutable borrowed content
52
53         //Functions and macros often dereference args implicitly
54         println!("x_1:{}", y_1: {}, z_1: {}", x_1, y_1, z_1);
55         //Output: x_1:5, y_1:10, z_1:25
56
57         //Data cannot be borrowed twice
58         ↪ error: println!("x:{}", y: {}, z: {}", x, y, z); //Compilation
59
60         //Cannot borrow x as immutable because
61         //it is already borrowed as mutable
62
63         //End of block, "z_1" is dropped
64         //"y_2" and "y_1" is dropped
65         //"x_1" is dropped
66     }
67     //Since the references were used, the original
68     //variables' values changed as well
69     println!("x: {}, y: {}, z: {}", x, y, z);
70     // "z" (including data on the heap), "y" and "x" are deallocated
71 }

```

```

71 fn demonstrate_moving() {
72     //Data can be owned only by exactly one variable.
73     //If it is being assigned to another variable, and type
74     //implements the 'Copy' trait, the new value is constructed.
75     //Otherwise, data ownership is transferred.
76     //i64 is 'Copy'
77     let x = 0i64;
78     let mut y = x;
79     y += 5;
80     println!("x:{}, y:{}", x, y);
81     //Output: x:0, y:5
82     //Vec<_> is not 'Copy'
83     let vec_1 = vec![0i64, 1, 2];
84     //Note that vec_2 is mutable, but vec_1 was not.
85     //Mutability is not a property of data,
86     //but a property of binding.
87     let mut vec_2 = vec_1;
88     vec_2.push(3);
89     //println!("vec_1: {:?}", vec_1); //Compilation error:
90     //Use of moved value
91     println!("vec_2: {:?}", vec_2);
92     //Output: vec_2:[0, 1, 2, 3]
93     //Function 'consume_vec' receives argument by value (as opposed
94     //to "by reference"), receiving the ownership, and consumes it.
95     consume_vec(vec_2);
96     //println!("vec_2: {:?}", vec_2); //Compilation error:
97     //Use of moved value
98 }
99
100 fn main() {
101     demonstrate_borrowing_and_deallocation();
102     demonstrate_moving();
103 }

```

Listing 2: Memory Management Demonstration

### 3.3. Custom Data Types

Writing a complex program using only predefined types would be inconvenient. For that reason, a programmer is allowed to introduce additional types. Custom data types in Rust are created through either of two keywords:

- **struct** – usually used to define a classic C-like struct.
- **enum** – used to define an enumerated type.

Rust's structs are generally similar to those of C. Specifically, a struct in either of languages can be considered a list of variables of different types grouped together. However, the concepts are not completely identical. That is, structs in Rust can implement methods as well, which bears similarity to C++ classes. On the other hand, unlike C++ classes, Rust structs cannot inherit from each other, due to the fact that the language itself adopts the "composition over inheritance" principle.

Enumerations are more powerful in Rust than in most other programming languages that use the **enum** keyword. In particular, Rust enumerations can have data associated with each enumeral. In other words, enumeration in Rust is a *sum type* (also known as *tagged union*). Sum types are usually present in FP languages, e.g., Haskell and OCaml [16].

Example of creating custom data types is given in Listing 3. Idiomatic way of working with enumerations is described in subsection 3.4.

```

1  //Defining custom data structure with 2 fields
2  struct Vector2 {
3      x: f64,
4      y: f64,
5  }
6
7  //Defining methods of the struct
8  impl Vector2 {
9      //Rust does not have constructors
10     //By convention *static method* "new" is used
11     fn new(in_x: f64, in_y: f64) -> Vector2 {
12         return Vector2 {
13             x: in_x,
14             y: in_y,
15         };
16     }
17
18     //Method that does not mutate "self".
19     fn len(&self) -> f64 {
20         //Note the lack of semicolon
21         //Identical to 'return' statement
22         (self.x * self.x + self.y * self.y).sqrt()
23     }
24
25     //Method that mutates "self".
26     fn normalize(&mut self) {
27         let len = self.len();
28         self.x /= len;
29         self.y /= len;
30     }
31
32     //Method that consumes and drops "self".
33     fn into_tuple(self) -> (f64, f64) {
34         (self.x, self.y)
35     }
36 }
37
38 //Defining custom enumerable.
39 //In Rust enumerals can also store other types inside
40 enum Task {
41     Sum(i32, i32),
42     Print(String),
43     DoNothing
44 }
45
46 fn main() {
47     let mut my_vec_1 = Vector2 {
48         x: 3f64,
49         y: 4f64,
50     };
51     let my_vec_2 = Vector2::new(8f64, 6f64);
52     println!("Lengths are {} and {}", my_vec_1.len(), my_vec_2.len());
53     //Output: Lengths are 5 and 10
54
55     my_vec_1.normalize();
56     //Following line causes compiler error
57     //"my_vec_2" is immutable, but method "normalize"
58     //takes mutable reference to "self"
59     //my_vec_2.normalize();
60
61 }

```

```

62      //structs can be destructured
63      let Vector2{x: x_1, y: y_1} = my_vec_1;
64      //Shorthand notation
65      let Vector2{x, y} = my_vec_2;
66
67      println!("Destructured: ({}, {}) and ({}, {})", x_1, y_1, x, y);
68      //Output: Destructured: (0.6,0.8) and (8,6)
69
70      //Initializing 3 enumerables with different values
71      let enum_1 = Task::DoNothing;
72      let enum_2 = Task::Sum(3, 5);
73      let enum_3 = Task::Print("Hello".to_owned());
74      //enums' associated data can be destructured as well
75      //Conditional destructuring
76      if let Task::Print(msg) = enum_3 {
77          println!("{}", msg);
78          //Output: Hello
79      }
80  }

```

Listing 3: Complex Data Types Demonstration

### 3.4. Control Flow

To implement any sort of logic, control flow structures need to be used. On this matter rust does not differ drastically from widely used languages: it employs the **if-then-else** expressions, **match** expressions (similar to **switch-case** in C), **for** loops and **while** loops. Additionally, the **loop** keyword indicates an infinite loop; the compiler will be able to apply better optimizations to the **loop** repetitions than to the **while (true)**. However, the **do-while** loops are not present in Rust.

The programmer should acknowledge that Rust is an expression language (as opposed to a statement language), hence (almost) everything returns a value. Albeit, oftentimes the value returned is a *unit*, i.e., an empty tuple.

Mechanism of throwing and catching exceptions is used by numerous programming languages, but is not present in Rust, for the following reason [22]:

Exceptions complicate understanding of control-flow, they express validity/invalidity outside of the type system, and they interoperate poorly with multithreaded code (a major focus of Rust).

Instead, a function which may fail should return a sum type, which can be either the result (if the function was executed successfully) or an error (otherwise). The standard library provides two enumerables for that purpose: (1) **Option**<T>, enumerals of which are **Some**(T) and **None**, thus making it represent a nullable value, and (2) **Result**<T, E>, which can be either **Ok**<T> or **Err**<E>.

```

1  fn demonstrate_branching() {
2      //Unlike C, Rust does not require parentheses in conditionals
3      if true {
4          println!("It works!");
5      } else {
6          println!("This should not happen");
7      }
8      //Output: It works!
9
10     //Rust is an expression language (as opposed to statement
    ↪ language)
11     //So everything can be evaluated
12     let x = 3i64;
13     let y = 5i64;
14     //let max = if x >= y {x;} else {y}; //Compilation error:
15     //branches return values of different types
16
17     let max = if x >= y {x} else {y};
18
19     //Rust's analog of C switch\case statement
20     match max {
21         3 => println!("Three is greater than five!"),
22         5 => println!("Five is greater than three"),
23         //match enforces exhaustiveness checking.
24         //That is, every possible option should be checked.
25         //"- " means "anything else" in this context.
26         _ => {
27             println!("This should not be happening");
28             panic!();
29         }
30     }
31     //Output: Five is greater than three
32
33     //Can also be used as expression
34     let opt: Option<i64> = Some(10);
35     let num = match opt {
36         Some(x) => x,
37         None => 0,
38     };
39     //Enums Option and Result also implement method "unwrap"
40     //which is a shorthand for getting a value if it is Some/Ok and
    ↪ panicking otherwise
41     let res: Result<String, i64> = Ok("Something".to_owned());
42     println!("{}", res);
43     //Output: Ok("Something")
44     println!("{}", res.unwrap());
45     //Output: Something
46 }
47
48 fn demonstrate_looping() {
49     //infinite loop
50     loop {
51         //breaking out of innerwards-most loop
52         break;
53     }
54
55     //while loop
56     let mut i = 0i64;
57     while i < 5 {
58         i += 1;
59         if i == 3 {
60             //ignoring the rest of the loop body
61             //and going to next iteration
62             continue;
63         }
64         print!("{}", i);
65     }
66     println!("");
67     //Output: 1 2 4 5
68
69     //There are no 'do-while' loops, however
70
71     //For loops can iterate over anything that
72     //implements trait 'Iterator'
73
74     //Looping over range:

```

```

75     for j in 5..10 {
76         print!("{}", j);
77     }
78     println("");
79     //Output: 5 6 7 8 9
80
81     //Looping over arrays:
82     let mut nums = [-2i64, -1, 0, 1, 2];
83     let mut sum = 0i64;
84     let mut sum_of_squares = 0i64;
85
86     //Looping over immutable references:
87     for num in nums.iter() {
88         sum += *num;
89     }
90     println!("{}", sum);
91     //Output: 0
92
93     //Looping over mutable references:
94     for num in nums.iter_mut() {
95         *num *= *num;
96     }
97     println!("{}", nums);
98     //Output: [4, 1, 0, 1, 4]
99
100    //Looping and taking ownership. Consumes vec
101    for num in nums.into_iter() {
102        sum_of_squares += num;
103    }
104    println!("{}", sum_of_squares);
105    //Output: 10
106    //println!("{}", nums); //Compilation error:
107    //use of moved value
108 }
109
110 fn main() {
111     demonstrate_branching();
112     demonstrate_looping();
113 }

```

Listing 4: Control Flow Demonstration

### 3.5. Polymorphism

Polymorphism in Rust is achieved via traits and generics (functions and data types) with trait bounds. While generics may be familiar to the reader from other languages, and as such will not be covered in detail, traits deserve deeper explanation.

Schärli et al. [23] introduced traits in 2003 and described them as "Composable units of behaviour". They noticed that in Object-Oriented Programming classes play two competing roles: (1) *generator of instances*, and as such, class should be logically complete, and (2) *unit of reuse*, which should be as small as possible. Due to this observation, the team proposed the idea of traits, which are, when described in a simplified way, collections of required and provided methods. Each class (or, in case of Rust, struct) can implement any amount of traits. Each trait may depend on any amount of other traits. Traits do not specify and cannot access any state variables. Traits have a way to resolve name conflicts, therefore classes (or structs) can implement

traits which have methods with the same name. Listing 5 provides examples of working with traits in Rust.

It should also be noted, that traits often are considered similar to Haskell’s type-classes [16, 22]; this can be interpreted as another sign of FP influence on Rust.

```

1  //Trait declaration
2  trait MyTrait {
3      fn say_hello(&self);
4  }
5
6  trait MyOtherTrait {}
7
8  //Data type need to implement both MyTrait and MyOtherTrait
9  //to be able to implement MyDependentTrait
10 trait MyDependentTrait: MyTrait + MyOtherTrait {
11     fn required_method(&self);
12     fn say_hello(&self);
13     fn provided_method(&self) {
14         println!("provided_method");
15         println!("It is same for every type by default, but can be
16         ↪ redefined");
17     }
18 }
19
20 //Some traits can be derived by compiler
21 #[derive(Clone, Copy, Debug)]
22 struct Foo {
23     x: isize,
24 }
25
26 impl MyTrait for Foo {
27     fn say_hello(&self) {
28         println!("Hello from Foo");
29     }
30 }
31
32 #[derive(Clone, Copy, Debug)]
33 struct Bar<T: MyTrait> {
34     data: T,
35 }
36
37 impl<T: MyTrait> MyTrait for Bar<T> {
38     fn say_hello(&self) {
39         println!("Hello from Bar's implementation of MyTrait");
40     }
41 }
42
43 impl<T: MyTrait> MyOtherTrait for Bar<T> {}
44
45 impl<T: MyTrait> MyDependentTrait for Bar<T> {
46     fn required_method(&self) {
47         println!("Calling hello on self and on data from Bar's
48         ↪ required_method");
49         //self.say_hello(); //Name conflict
50         //Need to specify which trait's implementation exactly to
51         ↪ call
52         MyDependentTrait::say_hello(self);
53         self.data.say_hello();
54     }
55     fn say_hello(&self) {
56         println!("Hello from Bar's implementation of
57         ↪ MyDependentTrait");
58     }
59 }
60 //generic function
61 fn exec_say_hello<T: MyTrait>(greeter: &T) {
62     greeter.say_hello();
63 }
64
65 fn main() {

```

```

66         let foo = Foo{x: 0};
67         // "Bar" did not specify the stored type explicitly,
68         // but rather "any type that implement MyTrait".
69         // "Foo" fits this requirement.
70         let bar = Bar{data: Foo{x: 5}};
71         exec_say_hello(&foo);
72         // Output: Hello from Foo
73
74         // exec_say_hello only has "MyTrait" as a trait bound, so no name
→   conflict occurs
75         exec_say_hello(&bar);
76         // Output: Hello from Bar's implementation of MyTrait
77
78         // Calling say_hello on "bar" manually will lead to compilation
→   error
79
80         bar.required_method();
81         // Output:
82         // Calling hello on self and on data from Bar's required_method
83         // Hello from Bar's implementation of MyDependentTrait
84         // Hello from Foo
85     }

```

Listing 5: Polymorphism Demonstration

## 4. Implementation

### 4.1. Task Overview

As it was outlined in the Introduction (see 1.3), two programs needed to be done in the scope of this study. The primary concern is achieving high performance. The reason for that is the exponential growth of the number of possible extensions. Provided that the modern CPUs have multiple cores, the programs should parallelize the workload for the best performance.

The first program (hereafter mentioned as "Extender") attempts to create a word of maximum possible length. It should also be able to continue working on words from input file, rather than start anew each time. The input file for this program is a list of words that will be used as prefixes, separated by line breaks.

The second program (hereafter "Classifier") marks words from input file as "satisfactory" or "unsatisfactory" based on the amount of symbols that can be appended and prepended (not simultaneously). The idea behind is to create a list of possible parts from which considerably longer words can be constructed. It might also be that additional restrictions need to be imposed in order to achieve better sorting accuracy. Therefore the program will not only prohibit factors which can be divided into two parts with equal number of occurrences of letters. Words with factors that can be divided into two parts the difference between which equals a user defined value (hereafter

referenced as "template vector") will be avoided as well. By providing only template vector  $(0,0,0)$ , the user reduces the task to avoiding abelian squares. The input file should be in the following format:

1. Number of template vectors ( $n$ ).
2. Template vectors; consists of  $n$  lines. The individual elements of each template vector are separated by spaces.
3. Space separated numbers  $l$  and  $r$  representing by how many letters the words should be extended to the left and to the right.
4. Words that should be extended. Each word starts on a new line.

Despite the notable differences between these two programs, a considerable amount of code written for one of them can be reused by the other. The decision was made to implement the Extender first and later to modify it into Classifier. The full code of the programs can be found in appendices 1, 2, 3, 4, 5 and 6.

#### 4.2. Single-Threaded Code

Instead of implementing the multithreaded code immediately, the author decided to write a single-threaded version of the Extender program and parallelize it afterward. This section covers the programming of the single-threaded version of the Extender. A word will be represented as a list of unsigned 8-bit integer values, that is, **Vec<u8>**.

One of the most important pieces of code in this program is the function that checks if a word is AA2F or not. Listing 6 demonstrates the most straightforward approach to implementing this function. For better readability, calculation of the Parikh vector of a word or a factor was moved into independent function.

```

1  //Parikh vector has 3 elements because with are working with alphabet of 3
   ↪ characters
2  fn calculate_parikh_vector(factor: &[u8]) -> [usize; 3] {
3      let mut result = [0; 3];
4      for letter in factor.iter() {
5          result[*letter as usize] += 1;
6      }
7      result
8  }
9
10 fn is_aa2f_full_check(word: &Vec<u8>) -> bool {
11     let l = word.len();
12     //words of length lesser than 4 are always AA2F
13     for part_1 in 4..(l + 1) {
14         for window in word.windows(part_1) {

```

```

15         //factors of odd length cannot be split into halves
16         if part_1 % 2 == 1 {
17             continue;
18         }
19         let pv_1 = calculate_parikh_vector(&window[0..(part_1 / 2)]);
20         let pv_2 = calculate_parikh_vector(&window[(part_1 /
    ↪ 2)..part_1]);
21         if pv_1 == pv_2 {
22             return false;
23         }
24     }
25     true
26 }
27 }

```

Listing 6: AA2F Full Check

As it was noted earlier in the thesis, the number of words grows exponentially as the length increases, therefore narrowing down the search field is advised. This can be done by checking only those words that are created by appending a single character to a word that is already known to be AA2F. To do so intermediate results need to be stored. The infinite loop that acts in this way is can be found in Listing 7. It also demonstrates how creation of words with given prefix can be done.

```

1  pub fn word_to_string(word: &Vec<u8>) -> String {
2      let utf: Vec<u8> = word.iter().map(|&x| x + 97).collect();
3      String::from_utf8(utf).expect(&*format!("Failed at line {}", line!()))
4  }
5
6  fn spawn_children(parent: &Vec<u8>) -> Vec<Vec<u8>> {
7      let mut result = Vec::with_capacity(3);
8      for i in 0..3 {
9          let mut new_word: Vec<u8> = Vec::with_capacity(parent.len() + 1);
10         new_word.clone_from(parent);
11         new_word.push(i as u8);
12         result.push(new_word);
13     }
14     result
15 }
16
17 fn generate_words(prefixes: &Vec<Vec<u8>>) {
18     let mut stack: Vec<Vec<u8>> = Vec::new();
19     stack.clone_from(prefixes);
20     //if necessary add empty word to begin generating words from scratch
21     if stack.len() == 0 {
22         stack.push(Vec::new());
23     }
24     let mut max_len: usize = 0;
25     //execute the loop until the stack is empty
26     while let Some(parent) = stack.pop() {
27         let children: Vec<Vec<u8>> = spawn_children(&parent);
28         let aa2f_children: Vec<Vec<u8>> = children
29             .into_iter()
30             .filter(|&wr| is_aa2f_full_check(wrd))
31             .collect();
32         for child in aa2f_children.into_iter() {
33             //print new word only if it is the longest known
34             if child.len() > max_len {
35                 max_len = child.len();
36                 println!("{}", child.len(), word_to_string(&child))
37             }
38             stack.push(child);
39         }
40     }
41 }

```

#### Listing 7: Single-Threaded Extender

It should be noted, that different data structures can be used to alter the order of word processing. For example, using a stack (a Last-In-First-Out data structure) will result in a depth-first search (that is, longest words first), while using a queue (a First-In-First-Out data structure) will result in a breadth-first search (shortest words first).

When utilizing such approach, it is known that the parent word of any word is AA2F and therefore any factor that was present in the parent need not be checked. This leaves us to check only suffixes of newly created words. By reducing the number of factors needed to be processed to check a single word a performance boost can be gained. The optimized AA2F checking function that uses this method is available in Listing 8.

Another optimization worth implementing is using cumulative Parikh vectors. Essentially, cumulative Parikh vector is a list of Parikh vectors of all prefixes of a word. Generation of cumulative Parikh vector has time complexity  $O(n)$ , where  $n$  is the size of the entire word. After this the Parikh vector of any factor of this word can be calculated by finding a difference of only two element of cumulative Parikh vector. This operation executes in constant time ( $O(1)$ ). Contrast with  $O(m)$  (where  $m$  is the factor's length;  $0 \leq m \leq n$ ) required to calculate a Parikh vector of each factor when using direct approach.

```

1  fn calculate_parikh_diff(p1: &[usize; 3], p2: &[usize; 3]) -> [usize; 3] {
2      let mut result = [0; 3];
3      for i in 0..p1.len() {
4          result[i] = p2[i] - p1[i];
5      }
6      result
7  }
8
9  fn calculate_cumul_pv(word: &Vec<u8>) -> Vec<[usize; 3]> {
10     let mut result = Vec::with_capacity(word.len() + 1);
11     result.push([0; 3]);
12     for i in 0..word.len() {
13         let mut new_elem = result[i].clone();
14         new_elem[word[i] as usize] += 1;
15         result.push(new_elem);
16     }
17     result
18 }
19
20 pub fn is_aa2f_partial_check(word: &Vec<u8>) -> bool {
21     if word.len() < 4 {
22         return true;
23     }
24     let l = word.len();
25     let cumul_pv = calculate_cumul_pv(word);

```

```

26     for i in 2..(1 / 2 + 1) {
27         let diff2 = calculate_parikh_diff(&cumul_pv[1 - i], &cumul_pv[1]);
28         let diff1 = calculate_parikh_diff(&cumul_pv[1 - 2 * i],
↪      &cumul_pv[1 - i]);
29         if diff1 == diff2 {
30             return false;
31         }
32     }
33     true
34 }

```

Listing 8: Optimized AA2F Check

The last modification to the `generate_words` function worth noting is using a closure function which would determine when to stop. This closure function evaluates each word before it is added to the queue and returns a boolean value representing whether generation should halt or continue.

```

1  fn generate_words<F>(prefixes: &Vec<Vec<u8>>, should_halt: F) -> String
2  where F: 'static + Fn(&Vec<u8>) -> bool{
3      let mut stack: Vec<Vec<u8>> = Vec::new();
4      stack.clone_from(prefixes);
5      //add empty word if necessary to begin generating words from scratch
6      if stack.len() == 0 {
7          stack.push(Vec::new());
8      }
9      let mut max_len: usize = 0;
10     //execute the loop until the stack is
11     while let Some(parent) = stack.pop() {
12         let children: Vec<Vec<u8>> = spawn_children(&parent);
13         let aa2f_children: Vec<Vec<u8>> = children
14             .into_iter()
15             .filter(|wr| is_aa2f_partial_check(wr))
16             .collect();
17         for child in aa2f_children.into_iter() {
18             //print new word only if it is the longest known
19             if child.len() > max_len {
20                 max_len = child.len();
21                 println!("{}", child.len(), word_to_string(&child))
22             }
23             if should_halt(&child) {
24                 return word_to_string(&child);
25             }
26             stack.push(child);
27         }
28     }
29     unreachable!()
30 }
31
32 fn main() {
33     //Attempt to generate and print a 10-character long word starting
↪    with "abc"
34     let final_word = generate_words(&vec![vec![0u8, 1, 2]], |word:
↪    &Vec<u8>| word.len() >= 10);
35     //Output: "Final word: abccbbbbc"
36     println!("Final word: {}", final_word);
37 }

```

Listing 9: Using Closure Function to Halt the Program

This concludes the list of functions necessary to implement the Extender program. The Classifier program relies on similar functions, however its analogs of AA2F checks are slightly more complex. As it was noted in subsection 4.1, instead of checking halves

of factors for equality, Classifier needs to perform more complex calculations. The Classifier's analogs of AA2F checking function need to see if factor can be divided into two such parts, that difference between them equals to a user-defined template vector. The code of template vector checking functions is shown below. Another thing to note about the Classifier program is that trying to extend a word to the left can be implemented as trying to extend reversed word to the right.

```

1  pub fn is_template_vector_free_full_check(word: &Word, templates:
    ↪ &Vec<[isize; cfg::CARDINALITY]>) -> bool {
2      let l = word.len();
3      //it is possible to shorten the full_check by calling partial_check
    ↪ from it
4      for part_len in 1..(l + 1) {
5          for window in word.windows(part_len) {
6              if !is_template_vector_free_partial_check(&window.to_vec(),
    ↪ templates) {
7                  return false;
8              }
9          }
10     }
11     true
12 }
13
14 pub fn is_template_vector_free_partial_check(word: &Word, templates:
    ↪ &Vec<[isize; cfg::CARDINALITY]>) -> bool {
15     let l = word.len() as isize;
16     let cumul_pv = calculate_cumul_pv(word);
17     for templ_vec in templates.iter() {
18         //norm of the template vector represents by how many
    ↪ character
19         //should one part be longer than the other
20         let norm = templ_vec.iter()
21             .fold(0, |acc, &x| acc + x);
22         let mut i: isize = 1;
23         while (i * 2) as isize + norm <= l as isize {
24             let diff1 = parikh_diff(&cumul_pv[(l - 2 * i - norm) as
    ↪ usize], &cumul_pv[(l - i) as usize]);
25             let diff2 = parikh_diff(&cumul_pv[(l - i) as usize],
    ↪ &cumul_pv[l as usize]);+9
26             let mut are_eq = true;
27             for j in 0..cfg::CARDINALITY {
28                 if diff1[j] - diff2[j] != templ_vec[j] {
29                     are_eq = false;
30                 }
31             }
32             if are_eq {
33                 return false;
34             }
35             i += 1;
36         }
37     }
38     true
39 }

```

Listing 10: Functions Checking if Word is Template Vector Free

Listing 10, which provides the code of template-checking functions used by the Classifier, concludes this subsection. The code used to process command line parameters and read file input provides no interest in the scope of this thesis. Nevertheless, it can be found in Appendices 1 and 4 for the Extender and the Classifier respectively.

### 4.3. Parallelization

The next step after writing a single-threaded program is to make full use of modern multi-core CPUs by parallelizing the program. Two major problems needed to be solved: how to communicate between threads and how to organize their work in the optimal manner.

#### 4.3.1. Thread Communication

Because the task requires constant updates on the current state (i.e., which words need to be tested), the communication between threads is requisite. Two primary methods of achieving this exist in Rust: communication over channels and communication by sharing memory.

Channels in Rust are one-way "multi-producer, single-consumer FIFO queue communication primitives" [24]. They can transfer ownership of almost any data type without any preparations required. To be more precise, data types implementing trait **Send** can have their ownership transferred between threads in a safe manner. The trait is derived by compiler automatically when appropriate (that is, when all fields of a data structure implement **Send** themselves). However using channels for two-way communication is problematic. It can be emulated to some extent by using two one-way channels, but such a solution can be hard for a programmer to write and, later, to read.

Communicating by sharing memory has stricter requirements: shared data should implement both **Send** and **Sync** traits. **Sync** trait marks data that can be safely shared between threads. It can be formally expressed as "a type **T** is **Sync** if **&T** is thread-safe" [24]. **Sync** is not as common as **Send**. The simplest solution is to wrap data type in a **Mutex** (mutual exclusion device) which will implement both **Send** and **Sync** if wrapped type implements **Send**. However, the **Mutex** itself is an owned data type and as such cannot be easily shared; its only purpose is to prevent simultaneous access to the underlying data. To overcome this problem, the **Mutex** itself should be wrapped in an **Arc** (atomically reference counted wrapper) to make it possible to share data without violating Rust's memory model. The final type can be denoted as **Arc<Mutex<T>>**.

To determine which of those approaches results in better performance, a test was conducted. In this test the master thread was distributing tasks between worker threads. The time needed to complete the tasks was measured for the single-threaded

code, for the multithreaded code that used channels and for the multithreaded code that used shared memory.

The testing code also employs bulking: transferring more than one task at a time. Such technique can reduce the communication overhead to some extent. Listing 11 displays the code that was used to measure the performance of different communication methods. For the sake of simplicity, the computational task being parallelized was unrelated to AA2F research. The program attempted to run 4 threads simultaneously. The tests were conducted on a 2-core 4-thread (due to the Intel Hyper-Threading Technology) CPU. The full code is available in Appendix 7.

```

1  fn test_shared_memory() {
2      //global task queue that is shared between threads
3      let numbers_to_process: Arc<Mutex<Vec<i64>>> =
4      ↪ Arc::new(Mutex::new((1..NUM_COUNT+1).collect()));
5      let mut sum = 0i64;
6      let start: u64 = precise_time_ns();
7      let mut threads: Vec<JoinHandle<_>> =
8      ↪ Vec::with_capacity(SPAWNED_THREAD_COUNT);
9      for _ in 0..SPAWNED_THREAD_COUNT {
10         let nums = numbers_to_process.clone();
11         let handle = spawn(move || {
12             let mut local_sum = 0i64;
13             //local task queue in which bulked tasks are stored
14             let mut task_queue: Vec<i64> = Vec::with_capacity(BULK_SIZE);
15             loop {
16                 if task_queue.len() > 0 {
17                     let num = task_queue.pop().expect("Pop result should
18                     ↪ not be empty");
19                     //collatz_length is the computation-heavy function
20                     local_sum += collatz_length(num);
21                 } else {
22                     let mut lock = nums.lock().expect("Mutex was
23                     ↪ poisoned");
24                     if lock.len() == 0 {
25                         return local_sum;
26                     } else {
27                         for _ in 0..min(lock.len(), BULK_SIZE) {
28                             task_queue.push(lock.pop().unwrap());
29                         }
30                     }
31                 }
32             }
33         });
34         threads.push(handle);
35     }
36     for join_handle in threads {
37         sum += join_handle.join().unwrap();
38     }
39     let end: u64 = precise_time_ns();
40     println!("Sum is: {:?}", sum);
41     println!("Took {} ns to compute in parallel using shared memory with
42     ↪ bulk size of {}", end - start, BULK_SIZE);
43 }
44
45 fn test_channels() {
46     //global task queue
47     let mut numbers_to_process: Vec<i64> = (1..NUM_COUNT+1).collect();
48     let mut sum = 0i64;
49     let mut threads: Vec<JoinHandle<_>> =
50     ↪ Vec::with_capacity(SPAWNED_THREAD_COUNT);
51     let start: u64 = precise_time_ns();

```

```

48 //the channel via which the main thread receives results from working
↳ threads
49 //the Sender of the worker's channel is passed as well, so that the
↳ main thread
50 //can send new task to the worker.
51 let (main_tx, main_rx): (Sender<(Sender<Option<Vec<i64>>>, i64)>,
↳ Receiver<(Sender<Option<Vec<i64>>>, i64)>) = channel();
52 for _ in 0..SPAWNED_THREAD_COUNT {
53     let tx = main_tx.clone();
54     let handle = spawn(move || {
55         let (local_tx, local_rx): (Sender<Option<Vec<i64>>>,
↳ Receiver<Option<Vec<i64>>>) = channel();
56         tx.send( (local_tx.clone(), 0) );
57         while let Ok(new_tasks_opt) = local_rx.recv() {
58             if let Some(new_tasks) = new_tasks_opt {
59                 let mut local_sum = 0i64;
60                 //new_tasks is the Vec that was sent by the main
61                 thread
62                 //serves as the local task queue
63                 for i in new_tasks.iter() {
64                     local_sum += collatz_length(*i);
65                 }
66                 //send to the main thread the local channel's Sender
67                 //and results of computation
68                 tx.send( (local_tx.clone(), local_sum) );
69             } else {
70                 return;
71             }
72         }
73     });
74     threads.push(handle);
75 }
76 drop(main_tx);
77 while let Ok( (tx, collatz_result) ) = main_rx.recv() {
78     //receive the results from workers
79     sum += collatz_result;
80     if numbers_to_process.len() == 0 {
81         tx.send(None);
82     } else {
83         let mut data_bulk: Vec<i64> = Vec::with_capacity(BULK_SIZE);
84         for _ in 0..min(BULK_SIZE, numbers_to_process.len()) {
85             data_bulk.push(numbers_to_process.pop().unwrap());
86         }
87         //and send new tasks to worker if there were some left
88         tx.send(Some(data_bulk));
89     }
90 }
91 }
92 for join_handle in threads {
93     join_handle.join();
94 }
95 }
96 let end: u64 = precise_time_ns();
97 println!("Sum is: {:?}" , sum);
98 println!("Took {} ns to compute in parallel using channels with bulk
99 size of {}", end - start, BULK_SIZE);
100 }

```

Listing 11: Testing Tread Communication Methods

Multiple runs with different bulk sizes were conducted; the details can be found in Appendix 8. Each run provided results about the time taken by the shared memory implementation, by the channels implementation and, as a baseline, by the serial program. On the figure below the reader can see the chart showing how many times faster the parallelized versions were than the serial one on average.

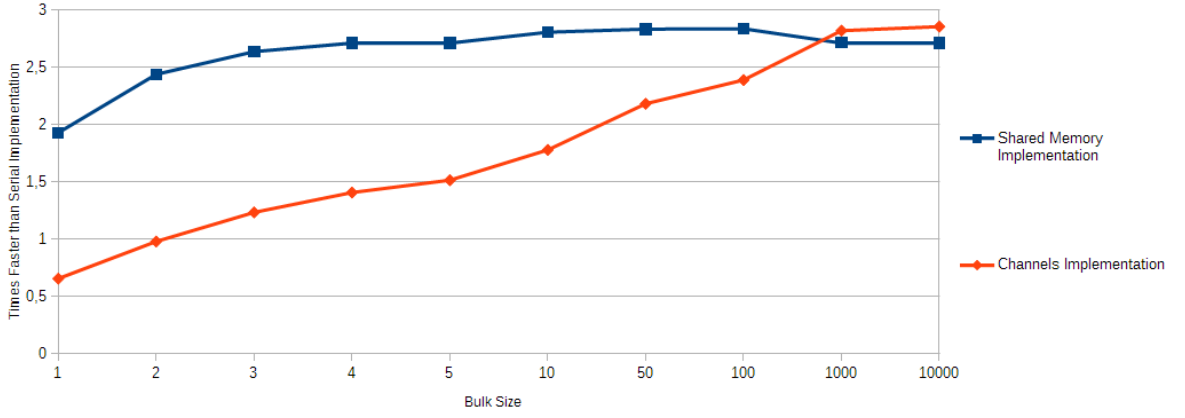


Figure 2: Comparison of Performance of Thread Communications Methods

It can be seen that the channels implementation has comparable performance to that of the shared memory implementation only at large bulk sizes. Furthermore, when only one task at a time was transferred, the channels version had lower performance than the serial program. Therefore the choice was between employing bulking together with using channels to communicate between threads, and using shared memory. However, using bulking in the either of programs developed in the scope of this project is unadvised. The reason is that such model of communication can lead to a single thread exhausting the task queue at low queue sizes and, consequently, to the distribution of the tasks in a suboptimal manner. As a result, the shared memory model was chosen.

#### 4.3.2. Generator Structure

At the core of both programs lies the "Generator" **struct** with minor differences. This custom data structure only has 2 or 3 fields:

##### 1. **task\_queue**

Stores words that are awaiting to be processed. Implemented as Binary Heap and therefore acts as a Priority Queue, which allows it to find the best word based on some criteria. The elements of Binary Heap need to implement **Ord** (ordering) trait so that Binary Heap is able to sort elements. To solve this, the custom wrapper for words needed to be written.

##### 2. **thread\_number**

Specifies how many additional worker threads this instance of the Generator is allowed to spawn.

### 3. `template_vecs`

Is a list (**Vec**) of template vector that need to be avoided. Exclusive to Classifier program.

Listing 12 contains the definition of the Generator **struct** as well as implementation of its "main" method (taken from the Classifier program). Note that the **generate\_until** method heavily resembles the **generate\_words** from Listing 9, but includes multiple mutex locks needed to keep the threads synchronized. In the Extender program the method is to some extent different and needs to employ atomic access operations as well (see Appendix 2).

```

1  #[derive(Debug)]
2  pub struct Generator {
3      task_queue: Arc<Mutex<BinaryHeap<Task>>>,
4      max_len: Arc<atomic::AtomicUsize>,
5      thread_number: usize,
6      template_vecs: Vec<isize; cfg::CARDINALITY>,
7  }
8
9  impl Generator {
10     pub fn generate_until<F>(&mut self, should_halt: F) -> String
11     where F: 'static + Send + Sync + Fn(&WordWrapper) -> bool {
12         let mut threads = Vec::with_capacity(self.thread_number);
13         let thread_number = self.thread_number;
14         let arced_should_halt = Arc::new(should_halt);
15         for _ in 0..thread_number {
16             let mut missed_pops = 0i32;
17             let queue = self.task_queue.clone();
18             let max_len = self.max_len.clone();
19             let should_halt = arced_should_halt.clone();
20             let local_templates = self.template_vecs.clone();
21             let new_thread = thread::spawn(move || {
22                 loop {
23                     let current_word: WordWrapper;
24                     {
25                         let mut queue_lock =
26                         ↪ queue.lock().expect(&format!("Failed at line {}", line!()));
27                         match queue_lock.pop() {
28                             Some(task) => {
29                                 missed_pops = 0;
30                                 match task {
31                                     Task::ProcessWord(word) =>
32                                     ↪ current_word = word,
33                                     Task::Halt => return None,
34                                 }
35                             }
36                             None => {
37                                 //if too many "misses" happen,
38                                 than the queue is simply empty and generator is stuck
39                                 if missed_pops > 1_000_000 {
40                                     return None;
41                                 } else {
42                                     missed_pops += 1;
43                                 }
44                                 continue;
45                             }
46                         }
47                     }
48                 }
49             });
50             threads.push(new_thread);
51         }
52     }
53 }

```

```

43         }
44         let children: Vec<WordWrapper> =
45     ↪ current_word.spawn_children()
46         .into_iter()
47         .filter(|wrd|
48     ↪ is_template_vector_free_partial_check(wrd, &local_templates))
49         .collect();
50         let mut queue_lock =
51     ↪ queue.lock().expect(&format!("Failed at line {}", line!()));
52         for child in children.iter() {
53             if should_halt(child) {
54                 for _ in 1..thread_number {
55                     queue_lock.push(Task::Halt);
56                 }
57                 return Some(word_to_string(child));
58             }
59             for child in children.into_iter() {
60                 queue_lock.push(Task::ProcessWord(child));
61             }
62         });
63         threads.push(new_thread);
64     }
65     let mut result: Option<String> = None;
66     for thread in threads {
67         if let Some(res) = thread.join().expect("Could not
68     ↪ join thread") {
69             result = Some(res);
70         }
71         //return empty string if the generator stopped by
72     ↪ exhausting its queue
73         result.unwrap_or("").to_owned()
74     }
75 }
76 }

```

Listing 12: Generator Definition and Implementation

## 5. Discussion

The thesis presents the process of development of two high-performance programs in a relatively young programming language. The application of the programs hopefully can advance the ongoing research of almost abelian square-free word on Three letters.

Before the implementation phase could begin, the author needed to conclude extensive research both in the mathematical and technical areas. In the technical research information needed to be acquired about the unusual concepts utilized by the Rust programming language (e.g. memory model and traits).

It can also be noted that the author's previous Extender program, written in JavaScript, utilized the similar architecture and managed to generate an almost abelian square-free word of length 15796 (compare with previous longest known word having the length of about 3150). This new long almost abelian square-free word can be found

in Appendix 9. Hopefully, the program written in Rust can provide even better result due to more efficient thread communication and the language itself being closer to hardware.

The research continues, currently the approach using template vector is more focused when compared to the direct generation of words. If the need arises, the source codes of the programs (which can be found at <https://bitbucket.org/AlGvrl/aa2f-rs/src>) should be easy to modify so that they will be more applicable to future research tasks.

## References

- [1] A. Thue, Über unendliche Zeichenreihen, Norske Vid. Selsk. Skr. I. Mat. Nat. Kl. Christiania 7 (1906) 1–22.
- [2] P. Erdős, Some unsolved problems, Magyar Tud. Akad. Mat. Kutat Int. Kzl 6 (1961) 221–254.  
URL <http://www.emis.ams.org/classics/Erdos/cit/87411003.htm>
- [3] P. A. Pleasants, Non-repetitive sequences, in: Mathematical Proceedings of the Cambridge Philosophical Society, Vol. 68, Cambridge Univ Press, 1970, pp. 267–274.  
URL [http://journals.cambridge.org/abstract\\_S0305004100046077](http://journals.cambridge.org/abstract_S0305004100046077)
- [4] R. C. Etringer, D. E. Jackson, J. A. Schatz, On nonrepetitive sequences, Journal of Combinatorial Theory 16 (1974) 159–164.
- [5] F. M. Dekking, Strongly non-repetitive sequences and progression-free sets, Journal of Combinatorial Theory, Series A 27 (2) (1979) 181–185.  
doi:10.1016/0097-3165(79)90044-X.  
URL <http://www.sciencedirect.com/science/article/pii/009731657990044X>
- [6] V. Keränen, Abelian squares are avoidable on 4 letters, in: W. Kuich (Ed.), Automata, Languages and Programming, no. 623 in Lecture Notes in Computer Science, Springer Berlin Heidelberg, 1992, pp. 41–52, doi: 10.1007/3-540-55719-9\_62.

URL [http://link.springer.com/chapter/10.1007/3-540-55719-9\\_62](http://link.springer.com/chapter/10.1007/3-540-55719-9_62)

[7] A. Carpi, On abelian power-free morphisms, *International Journal of Algebra and Computation* 3 (02) (1993) 151–167.

URL <http://www.worldscientific.com/doi/pdf/10.1142/S0218196793000123>

[8] A. Carpi, On the number of abelian square-free words on four letters, *Discrete Applied Mathematics* 81 (13) (1998) 155–167. doi:10.1016/S0166-218X(97)88002-X.

URL <http://www.sciencedirect.com/science/article/pii/S0166218X9788002X>

[9] V. Keränen, A powerful abelian square-free substitution over 4 letters, *Theoretical Computer Science* 410 (3840) (2009) 3893–3900. doi:10.1016/j.tcs.2009.05.027.

URL <http://www.sciencedirect.com/science/article/pii/S0304397509003892>

[10] The Rust programming language.

URL <https://www.rust-lang.org/>

[11] Which programs are fastest?

URL <http://benchmarksgame.alioth.debian.org/u64q/which-programs-are-fastest.html>

[12] Some benchmarks of different languages.

URL <https://github.com/kostya/benchmarks>

[13] The Rustonomicon.

URL <https://doc.rust-lang.org/nomicon/>

[14] J. Blandy, *Why Rust? Trustworthy, concurrent systems programming*, O'Reilly Media, 2015.

URL <http://www.oreilly.com/programming/free/files/why-rust.pdf>

- [15] J. Regehr, Race condition vs. data race (Mar. 2011).  
URL <http://blog.regehr.org/archives/490>
- [16] R. Poss, Rust for functional programmers (Jul. 2014).  
URL <http://science.rafael.poss.name/rust-for-functional-programmers.html>
- [17] The Rust Programming Language.  
URL <https://doc.rust-lang.org/book/>
- [18] Rust by example.  
URL <http://rustbyexample.com/>
- [19] The Rust reference.  
URL <https://doc.rust-lang.org/reference.html>
- [20] D. Grossman, G. Morrisett, T. Jim, M. Hicks, Y. Wang, J. Cheney, Region-based memory management in Cyclone, in: ACM Sigplan Notices, Vol. 37, ACM, 2002, pp. 282–293.  
URL <http://dl.acm.org/citation.cfm?id=512563>
- [21] N. Swamy, M. Hicks, G. Morrisett, D. Grossman, T. Jim, Safe manual memory management in Cyclone, Science of Computer Programming 62 (2) (2006) 122–144.  
URL <http://www.sciencedirect.com/science/article/pii/S0167642306000785>
- [22] Frequently Asked Questions. The Rust Programming Language.  
URL <https://www.rust-lang.org/faq.html>
- [23] N. Schärli, S. Ducasse, O. Nierstrasz, A. P. Black, Traits: Composable units of behaviour, in: ECOOP 2003 Object-Oriented Programming, Springer, 2003, pp. 248–274.  
URL [http://link.springer.com/chapter/10.1007/978-3-540-45070-2\\_12](http://link.springer.com/chapter/10.1007/978-3-540-45070-2_12)
- [24] Rust Standard Library API Reference.  
URL <https://doc.rust-lang.org/std/>

## Appendices

|   |                                                           |    |
|---|-----------------------------------------------------------|----|
| 1 | Contents of the src/main.rs File (Extender)               | 32 |
| 2 | Contents of the src/gen.rs File (Extender)                | 33 |
| 3 | Contents of the src/cfg.rs File (Extender)                | 38 |
| 4 | Contents of the src/main.rs File (Classifier)             | 38 |
| 5 | Contents of the src/gen.rs File (Classifier)              | 40 |
| 6 | Contents of the src/cfg.rs File (Classifier)              | 45 |
| 7 | The Thread Communication Methods Test                     | 45 |
| 8 | The Results of Thread Communication Testing in CSV Format | 47 |
| 9 | Almost Abelian Square Free Word of Length 15796           | 48 |

### 1. Contents of the src/main.rs File (Extender)

```
1  extern crate clap;
2
3  use clap::{Arg, App};
4  use std::fs::File;
5  use std::io::{BufRead, BufReader};
6
7  pub mod gen;
8  pub mod cfg;
9
10 fn main() {
11     let matches = App::new("AA2F Generator")
12         .about("Attempts to create longest AA2F word")
13         .arg(Arg::with_name("thread_num")
14             .short("t")
15             .long("threads")
16             .help("Number of worker threads to be spawned")
17             .default_value("4")
18             .takes_value(true))
19         .arg(Arg::with_name("input_path")
20             .short("i")
21             .long("input")
22             .help("(Optional) Path to the file containing word(s) that
↳ should be extended")
23             .takes_value(true))
24         .arg(Arg::with_name("backtrack")
25             .short("b")
26             .long("backtrack")
27             .help("Whether or not word(s) in the input file should be
↳ backtracked")
28             .requires("input_path"))
29         .get_matches();
30
31     let mut initial_words: Vec<String> = Vec::new();
32     let thread_num: usize = matches.value_of("thread_num").expect("Could
↳ not get thread number")
33         .parse::<usize>().expect("Thread number should be a positive
↳ number");
34     if let Some(input_path) = matches.value_of("input_path") {
35         let file = File::open(input_path).expect("Could not open the
↳ specified input file");
```

```

36         let reader = BufReader::new(&file);
37         for line in reader.lines() {
38             initial_words.push(line.expect("Something is very wrong with
↳ the input file"));
39         }
40     } else {
41         initial_words.push("a".to_string());
42     }
43     let should_backtrack: bool = matches.is_present("backtrack");
44     if should_backtrack {
45         let mut bt_groups: Vec<Vec<gen::Word>> = Vec::new();
46         for wrd in initial_words.iter() {
47             ↳ bt_groups.push(gen::generate_backtrack(&gen::string_to_word(wrd)));
48         }
49         for group in bt_groups {
50             for word in group.iter() {
51                 initial_words.push(gen::word_to_string(word));
52             }
53         }
54     }
55     println!("Starting gen");
56     let mut my_gen = gen::Generator::new(&initial_words, should_backtrack,
↳ thread_num);
57     my_gen.generate();
58 }
59 }

```

## 2. Contents of the src/gen.rs File (Extender)

```

1  use std::collections::binary_heap::BinaryHeap;
2  use std::sync::{Mutex, Arc};
3  use std::sync::atomic;
4  use std::thread;
5  use std::ops::{Deref, DerefMut};
6  use std::cmp::{PartialEq, Eq, PartialOrd, Ord, Ordering};
7  use cfg;
8
9  ///An alias for a 'Vec<u8>', represents a word.
10 pub type Word = Vec<u8>;
11
12 ///A wrapper for a Word, which implements Cmp based on a given heuristic.
13 ///Internally caches the result of [priority-calculating
14 ↳ function](fn.calculate_comparison_key.html).
15 ///Wrapper can be automatically dereferenced to the wrapped 'Word' under
16 ↳ some circumstances.
17 #[derive(Debug)]
18 pub struct WordWrapper {
19     data: Word,
20     comparison_key: i64,
21 }
22
23 ///Represents the action that a thread should execute.
24 #[derive(Debug)]
25 pub enum Task {
26     ///Spawn word's children and push those of them that are AA2F back to
27     ↳ the queue.
28     ProcessWord(WordWrapper),
29     ///Kill the thread.
30     Halt
31 }
32
33 impl Deref for WordWrapper {
34     type Target = Word;
35     fn deref(&self) -> &Word {
36         &self.data
37     }
38 }

```

```

36 }
37
38 impl DerefMut for WordWrapper {
39     fn deref_mut(&mut self) -> &mut Word {
40         &mut self.data
41     }
42 }
43
44 impl PartialEq for WordWrapper {
45     fn eq(&self, other: &Self) -> bool {
46         self.comparison_key == other.comparison_key
47     }
48 }
49
50 impl Eq for WordWrapper {
51 }
52
53 impl PartialOrd for WordWrapper {
54     fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
55         self.comparison_key.partial_cmp(&other.comparison_key)
56     }
57 }
58
59 impl Ord for WordWrapper {
60     fn cmp(&self, other: &Self) -> Ordering {
61         self.comparison_key.cmp(&other.comparison_key)
62     }
63 }
64
65 impl WordWrapper {
66     pub fn new(word: Word) -> WordWrapper {
67         WordWrapper{
68             comparison_key: calculate_comparison_key(&word),
69             data: word,
70         }
71     }
72
73     pub fn spawn_children(&self) -> Vec<WordWrapper> {
74         let mut result = Vec::with_capacity(cfg::CARDINALITY);
75         for i in 0..cfg::CARDINALITY {
76             let mut new_word: Word = Vec::with_capacity(self.data.len() +
77                 ↪ 1);
78             new_word.clone_from(&self.data);
79             new_word.push(i as u8);
80             result.push(WordWrapper::new(new_word));
81         }
82         result
83     }
84 }
85
86 pub fn calculate_comparison_key(word: &Word) -> i64 {
87     let length = word.len() as i64;
88     let mean: i64 = length / (cfg::CARDINALITY as i64);
89     let mut unbalancedness = 0i64;
90     unbalancedness -= length % (cfg::CARDINALITY as i64);
91     let parikh = calculate_parikh_vector(word);
92     for i in parikh.iter() {
93         unbalancedness += (*i as i64 - mean).abs() as i64;
94     }
95     length * length - unbalancedness * unbalancedness * unbalancedness *
96     ↪ 27 / 8 / length
97 }
98
99 impl PartialEq for Task {
100     fn eq(&self, other: &Self) -> bool {
101         match (self, other) {
102             (&Task::ProcessWord(ref word_1), &Task::ProcessWord(ref
103                 ↪ word_2)) => {
104                 word_1 == word_2
105             },
106             (&Task::Halt, &Task::Halt) => true,
107             _ => false
108         }
109     }
110 }
111
112 impl Eq for Task {

```

```

113 }
114
115 impl PartialOrd for Task {
116     fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
117         match (self, other) {
118             (&Task::ProcessWord(ref w_1), &Task::ProcessWord(ref w_2)) =>
119                 ↪ w_1.partial_cmp(&w_2),
120             (&Task::Halt, &Task::Halt) => Some(Ordering::Equal),
121             (&Task::Halt, _) => Some(Ordering::Greater),
122             (_, &Task::Halt) => Some(Ordering::Less),
123             // _ => panic!()
124         }
125     }
126 }
127
128 impl Ord for Task {
129     fn cmp(&self, other: &Self) -> Ordering {
130         match (self, other) {
131             (&Task::ProcessWord(ref w_1), &Task::ProcessWord(ref w_2)) =>
132                 ↪ w_1.cmp(&w_2),
133             (&Task::Halt, &Task::Halt) => Ordering::Equal,
134             (&Task::Halt, _) => Ordering::Greater,
135             (_, &Task::Halt) => Ordering::Less,
136             // _ => panic!()
137         }
138     }
139 }
140
141 #[derive(Debug)]
142 pub struct Generator {
143     task_queue: Arc<Mutex<BinaryHeap<Task>>>,
144     max_len: Arc<atomic::AtomicUsize>,
145     thread_number: usize,
146 }
147
148 impl Generator {
149     pub fn new(strings: &[String], backtrack_input: bool, threads: usize)
150         ↪ -> Generator {
151         let max_len: Arc<atomic::AtomicUsize> =
152         ↪ Arc::new(atomic::ATOMIC_USIZE_INIT);
153         let queue: Arc<Mutex<BinaryHeap<Task>>> =
154         ↪ Arc::new(Mutex::new(BinaryHeap::with_capacity(cfg::HEAP_CAPACITY)));
155         {
156             let mut queue_lock = queue.lock().expect(&format!("Failed at
157             ↪ line {}", line!()));
158             for s in strings {
159                 let word = string_to_word(s);
160                 if backtrack_input {
161                     let bt_words = generate_backtrack(&word);
162                     for bt_word in bt_words.into_iter() {
163                         ↪ queue_lock.push(Task::ProcessWord(WordWrapper::new(bt_word)));
164                     }
165                 } else {
166                     ↪ queue_lock.push(Task::ProcessWord(WordWrapper::new(word)));
167                 }
168             }
169         }
170         Generator {
171             max_len: max_len,
172             task_queue: queue,
173             thread_number: threads,
174         }
175     }
176
177     pub fn generate(&mut self) {
178         self.generate_until(|_| &WordWrapper| false);
179     }
180
181     pub fn generate_until<F>(&mut self, should_halt: F) -> String
182     where F: 'static + Send + Sync + Fn(&WordWrapper) -> bool {
183         let mut threads = Vec::with_capacity(self.thread_number);
184         let thread_number = self.thread_number;
185         let arced_should_halt = Arc::new(should_halt);
186         for _ in 0..thread_number {

```

```

182         let mut missed_pops = 0i32;
183         let queue = self.task_queue.clone();
184         let max_len = self.max_len.clone();
185         let should_halt = arced_should_halt.clone();
186         let new_thread = thread::spawn(move || {
187             loop {
188                 let current_word: WordWrapper;
189                 {
190                     let mut queue_lock =
191 ↪ queue.lock().expect(&format!("Failed at line {}", line!()));
192                     match queue_lock.pop() {
193                         Some(task) => {
194                             missed_pops = 0;
195                             match task {
196 ↪ current_word = word,
197                                     Task::ProcessWord(word) =>
198                                         Task::Halt => return None,
199                                     }
200                                     None => {
201                                         if missed_pops > 1_000_000 {
202                                             return None;
203                                         } else {
204                                             missed_pops += 1;
205                                         }
206                                         //thread::sleep_ms(5);
207                                         continue;
208                                     }
209                                 }
210                                 let children: Vec<WordWrapper> =
211 ↪ current_word.spawn_children()
212                                     .into_iter()
213                                     .filter(|wrd| is_aa2f_partial_check(wrd))
214                                     .collect();
215                                     //Comment out from here...
216                                     for child in children.iter() {
217                                         if child.len() >
218 ↪ max_len.load(atomic::Ordering::Relaxed) {
219                                             max_len.store(child.len(),
220 ↪ atomic::Ordering::Relaxed);
221                                             println!("{}", child.len(),
222 ↪ word_to_string(child));
223                                         }
224                                         //...to here to avoid printing words
225                                         let mut queue_lock =
226 ↪ queue.lock().expect(&format!("Failed at line {}", line!()));
227                                         for child in children.iter() {
228                                             if should_halt(child) {
229                                                 for _ in 1..thread_number {
230                                                     queue_lock.push(Task::Halt);
231                                                 }
232                                                 return Some(word_to_string(child));
233                                             }
234                                         }
235                                         for child in children.into_iter() {
236                                             queue_lock.push(Task::ProcessWord(child));
237                                         }
238                                     }
239                                     });
240                                     threads.push(new_thread);
241                                     let mut result: Option<String> = None;
242                                     for thread in threads {
243                                         if let Some(res) = thread.join().expect("Could not join
244 ↪ thread") {
245                                             result = Some(res);
246                                         }
247                                     }
248                                     result.unwrap_or("").to_owned()
249                                 }
250                             }
251                         }
252                     }
253                 }
254             }
255         });
256     }

```

```

248
249 fn calculate_parikh_chain(word: &Word) -> Vec<usize; cfg::CARDINALITY> {
250     let mut result = Vec::with_capacity(word.len() + 1);
251     result.push([0; cfg::CARDINALITY]);
252     for i in 0..word.len() {
253         let mut new_elem = result[i].clone();
254         new_elem[word[i] as usize] += 1;
255         result.push(new_elem);
256     }
257     result
258 }
259
260 pub fn calculate_parikh_vector(word: &Word) -> [usize; cfg::CARDINALITY] {
261     let mut result = [0; cfg::CARDINALITY];
262     for letter in word.iter() {
263         result[*letter as usize] += 1;
264     }
265     result
266 }
267
268 fn parikh_diff(p1: &[usize; cfg::CARDINALITY], p2: &[usize;
    ↪   cfg::CARDINALITY]) -> [usize; cfg::CARDINALITY] {
269     let mut result = [0; cfg::CARDINALITY];
270     for i in 0..p1.len() {
271         result[i] = p2[i] - p1[i];
272     }
273     result
274 }
275
276 pub fn is_aa2f_partial_check(word: &Word) -> bool {
277     if word.len() < 4 {
278         return true;
279     }
280     let l = word.len();
281     let parikh_chain = calculate_parikh_chain(word);
282     for i in 2..(l / 2 + 1) {
283         let diff2 = parikh_diff(&parikh_chain[l - i], &parikh_chain[l]);
284         let diff1 = parikh_diff(&parikh_chain[l - 2 * i], &parikh_chain[l
    ↪   - i]);
285         if diff1 == diff2 {
286             return false;
287         }
288     }
289     true
290 }
291
292 pub fn is_aa2f_full_check(word: &Word) -> bool {
293     let l = word.len();
294     for part_len in 1..(l + 1) {
295         for window in word.windows(part_len) {
296             if !is_aa2f_partial_check(&window.to_vec()) {
297                 return false;
298             }
299         }
300     }
301     true
302 }
303
304 pub fn word_to_string(word: &Word) -> String {
305     let utf: Vec<u8> = word.iter().map(|&x| x + 97).collect();
306     String::from_utf8(utf).expect(&*format!("Failed at line {}", line!()))
307 }
308
309 pub fn string_to_word(in_str: &str) -> Word {
310     in_str.as_bytes().iter().map(|x| x - 97).collect()
311 }
312
313 pub fn generate_backtrack(word: &Word) -> Vec<Word> {
314     let mut result = Vec::with_capacity(word.len() - 1);
315     for i in (1..word.len()).rev() {
316         result.push(word[0..i].to_vec());
317     }
318     result
319 }

```

### 3. Contents of the src/cfg.rs File (Extender)

```
1 pub const CARDINALITY: usize = 3;
2 pub const HEAP_CAPACITY: usize = 1_000_000;
```

### 4. Contents of the src/main.rs File (Classifier)

```
1 extern crate clap;
2
3 use clap::{Arg, App};
4 use std::fs::File;
5 use std::io::{BufRead, BufReader, Write};
6
7 pub mod gen;
8 pub mod cfg;
9
10 use gen::Generator;
11
12 fn main() {
13     let matches = App::new("AA2F Classifier")
14         .about("Classifies words into extendable and ")
15         .arg(Arg::with_name("thread_num")
16             .short("t")
17             .long("threads")
18             .help("Number of worker threads to be spawned")
19             .default_value("4")
20             .takes_value(true))
21         .arg(Arg::with_name("input_path")
22             .short("i")
23             .long("input")
24             .help("Path to the file containing word(s) that should be
25 ↪ classified")
26             .required(true)
27             .takes_value(true))
28         .arg(Arg::with_name("good_name")
29             .short("g")
30             .long("good")
31             .help("(Optional) Name of the file that will be used as an
32 ↪ output for \"good\" words")
33             .takes_value(true))
34         .arg(Arg::with_name("bad_name")
35             .short("b")
36             .long("bad")
37             .help("(Optional) Name of the file that will be used as an
38 ↪ output for \"bad\" words")
39             .takes_value(true))
40         .get_matches();
41
42     let thread_num: usize = matches.value_of("thread_num").expect("Could
43 ↪ not ge thread number")
44         .parse::<usize>().expect("Thread number should be a positive
45 ↪ number");
46
47     let good_name = matches.value_of("good_name").unwrap_or("sfsg.txt");
48     let bad_name = matches.value_of("bad_name").unwrap_or("bad.txt");
49     let mut good_out = File::create(good_name).expect("Could not create
50 ↪ output file");
51     let mut bad_out = File::create(bad_name).expect("Could not create
52 ↪ output file");
53     let input_path = matches.value_of("input_path").expect("Could not
54 ↪ retrieve input file's name");
```

```

46 let input = File::open(input_path).expect("Could not open the input
47 file");
48 let reader = BufReader::new(&input);
49 let mut lines = reader.lines();
50 let temp_vec_num: usize = lines.next().expect("Iterator
51 error").expect("File input error")
52 .parse::<usize>().expect("Number parsing error");
53 let mut temp_vecs: Vec<isize; cfg::CARDINALITY> =
54 Vec::with_capacity(temp_vec_num);
55 good_out.write_all(temp_vec_num.to_string().as_bytes()).expect("Could
56 not write to the \"good\" file");
57 good_out.write_all("\n".as_bytes()).expect("Could not write to the
58 \"good\" file");
59 for _ in 0..temp_vec_num {
60 let line: String = lines.next().expect("Iterator
61 error").expect("File input error");
62 good_out.write_all(line.as_bytes()).expect("Could not write to the
63 \"good\" file");
64 good_out.write_all("\n".as_bytes()).expect("Could not write to the
65 \"good\" file");
66 let nums: Vec<isize> = line.split_whitespace()
67 .map(|num_str: &str| num_str.parse::<isize>().expect("Number
68 parsing error"))
69 .collect();
70 let mut temp_vec: [isize; cfg::CARDINALITY] = [0;
71 cfg::CARDINALITY];
72 for i in 0..temp_vec.len() {
73 temp_vec[i] = nums[i];
74 }
75 temp_vecs.push(temp_vec);
76 }
77 let extensions_str: String = lines.next().expect("Iterator
78 error").expect("File input error");
79 good_out.write_all(extensions_str.as_bytes()).expect("Could not write
80 to the \"good\" file");
81 good_out.write_all("\n".as_bytes()).expect("Could not write to the
82 \"good\" file");
83 let extensions: Vec<usize> = extensions_str
84 .split_whitespace()
85 .map(|num_str: &str| num_str.parse::<usize>().expect("Number
86 parsing error"))
87 .collect();
88 let extension_len_l: usize;
89 let extension_len_r: usize;
90 if extensions.len() != 2 {
91 panic!("Wrong input file format");
92 } else {
93 extension_len_l = extensions[0];
94 extension_len_r = extensions[1];
95 }
96 //only words should be left in the file at this point
97 for line in lines {
98 let word: String = line.expect("Could not read a word");
99 let reversed: String = word.chars().rev().collect();
100 let needed_len_l = word.len() + extension_len_l;
101 let needed_len_r = word.len() + extension_len_r;
102 let mut gen_1 = gen::Generator::new(&vec![word.clone()], false,
103 thread_num, temp_vecs.clone());
104 let right_res: String = gen_1.generate_until(move |x| x.len() >=
105 needed_len_r);
106 let can_be_ext_right = (right_res.len() > 0);
107 let mut gen_2 = gen::Generator::new(&vec![reversed], false,
108 thread_num, temp_vecs.clone());
109 let left_res: String = gen_2.generate_until(move |x| x.len() >=
110 needed_len_l);
111 let can_be_ext_left = (left_res.len() > 0);

```

```

97         if can_be_ext_right && can_be_ext_left {
98             good_out.write_all(left_res.chars().rev().collect:::<String>().as_bytes()).expect
↳ not write to the \"good\" file");
99             good_out.write_all("\n".as_bytes()).expect("Could not write to
↳ the \"good\" file");
100             good_out.write_all(right_res.as_bytes()).expect("Could not
↳ write to the \"good\" file");
101             good_out.write_all("\n".as_bytes()).expect("Could not write to
↳ the \"good\" file");
102         } else {
103             bad_out.write_all(word.as_bytes()).expect("Could not write to
↳ the \"bad\" file");
104             bad_out.write_all("\n".as_bytes()).expect("Could not write to
↳ the \"bad\" file");
105         }
106     }
107 }

```

## 5. Contents of the src/gen.rs File (Classifier)

```

1  use std::collections::binary_heap::BinaryHeap;
2  use std::sync::{Mutex, Arc};
3  use std::sync::atomic;
4  use std::thread;
5  use std::ops::{Deref, DerefMut};
6  use std::cmp::{PartialEq, Eq, PartialOrd, Ord, Ordering};
7  use cfg;
8
9  pub type Word = Vec<u8>;
10
11  #[derive(Debug)]
12  pub struct WordWrapper {
13      data: Word,
14      comparison_key: i64,
15  }
16
17  #[derive(Debug)]
18  pub enum Task {
19      ///Spawn word's children and push those of them that are AA2F back to
↳ the queue.
20      ProcessWord(WordWrapper),
21      ///Kill the thread.
22      Halt
23  }
24
25  impl Deref for WordWrapper {
26      type Target = Word;
27
28      fn deref(&self) -> &Word {
29          &self.data
30      }
31  }
32
33  impl DerefMut for WordWrapper {
34      fn deref_mut(&mut self) -> &mut Word {
35          &mut self.data
36      }
37  }
38
39  impl PartialEq for WordWrapper {
40      fn eq(&self, other: &Self) -> bool {
41          self.comparison_key == other.comparison_key
42      }
43  }
44
45  impl Eq for WordWrapper {
46  }
47
48

```

```

49 impl PartialOrd for WordWrapper {
50     fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
51         self.comparison_key.partial_cmp(&other.comparison_key)
52     }
53 }
54
55 impl Ord for WordWrapper {
56     fn cmp(&self, other: &Self) -> Ordering {
57         self.comparison_key.cmp(&other.comparison_key)
58     }
59 }
60
61 impl WordWrapper {
62     pub fn new(word: Word) -> WordWrapper {
63         WordWrapper{
64             comparison_key: calculate_comparison_key(&word),
65             data: word,
66         }
67     }
68
69     pub fn spawn_children(&self) -> Vec<WordWrapper> {
70         let mut result = Vec::with_capacity(cfg::CARDINALITY);
71         for i in 0..cfg::CARDINALITY {
72             let mut new_word: Word = Vec::with_capacity(self.data.len() +
73                 ↪ 1);
74             new_word.clone_from(&self.data);
75             new_word.push(i as u8);
76             result.push(WordWrapper::new(new_word));
77         }
78     }
79 }
80
81 pub fn calculate_comparison_key(word: &Word) -> i64 {
82     let length = word.len() as i64;
83     let mean: i64 = length / (cfg::CARDINALITY as i64);
84     let mut unbalancedness = 0i64;
85     unbalancedness -= length % (cfg::CARDINALITY as i64);
86     let parikh = calculate_parikh_vector(word);
87     for i in parikh.iter() {
88         unbalancedness += (*i as i64 - mean).abs() as i64;
89     }
90     length * length - unbalancedness * unbalancedness * unbalancedness *
91     ↪ 27 / 8 / length
92 }
93
94 impl PartialEq for Task {
95     fn eq(&self, other: &Self) -> bool {
96         match (self, other) {
97             (&Task::ProcessWord(ref word_1), &Task::ProcessWord(ref
98                 ↪ word_2)) => {
99                 word_1 == word_2
100             },
101             (&Task::Halt, &Task::Halt) => true,
102             _ => false
103         }
104     }
105 }
106
107 impl Eq for Task {}
108
109 impl PartialOrd for Task {
110     fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
111         match (self, other) {
112             (&Task::ProcessWord(ref w_1), &Task::ProcessWord(ref w_2)) =>
113             ↪ w_1.partial_cmp(&w_2),
114             (&Task::Halt, &Task::Halt) => Some(Ordering::Equal),
115             (&Task::Halt, _) => Some(Ordering::Greater),
116             (_, &Task::Halt) => Some(Ordering::Less),
117             // _ => panic!()
118         }
119     }
120 }
121

```

```

122 impl Ord for Task {
123     fn cmp(&self, other: &Self) -> Ordering {
124         match (self, other) {
125             (&Task::ProcessWord(ref w_1), &Task::ProcessWord(ref w_2)) =>
126                 ↪ w_1.cmp(&w_2),
127             (&Task::Halt, &Task::Halt) => Ordering::Equal,
128             (&Task::Halt, _) => Ordering::Greater,
129             (_, &Task::Halt) => Ordering::Less,
130             // _ => panic!()
131         }
132     }
133 }
134
135 #[derive(Debug)]
136 pub struct Generator {
137     task_queue: Arc<Mutex<BinaryHeap<Task>>>,
138     max_len: Arc<atomic::AtomicUsize>,
139     thread_number: usize,
140     template_vecs: Vec<[isize; cfg::CARDINALITY]>,
141 }
142
143 impl Generator {
144     pub fn new(strings: &[String], backtrack_input: bool, threads: usize,
145         ↪ templates: Vec<[isize; cfg::CARDINALITY]>) -> Generator {
146         let max_len: Arc<atomic::AtomicUsize> =
147             ↪ Arc::new(atomic::ATOMIC_USIZE_INIT);
148         let queue: Arc<Mutex<BinaryHeap<Task>>> =
149             ↪ Arc::new(Mutex::new(BinaryHeap::with_capacity(cfg::HEAP_CAPACITY)));
150         {
151             let mut queue_lock = queue.lock().expect(&format!("Failed at
152             ↪ line {}", line!()));
153             for s in strings {
154                 let word = string_to_word(s);
155                 if backtrack_input {
156                     let bt_words = generate_backtrack(&word);
157                     for bt_word in bt_words.into_iter() {
158                         ↪ queue_lock.push(Task::ProcessWord(WordWrapper::new(bt_word)));
159                     }
160                 } else {
161                     ↪ queue_lock.push(Task::ProcessWord(WordWrapper::new(word)));
162                 }
163             }
164         }
165         Generator {
166             max_len: max_len,
167             task_queue: queue,
168             thread_number: threads,
169             template_vecs: templates
170         }
171     }
172
173     pub fn generate(&mut self) -> String {
174         ↪ self.generate_until(|_| &WordWrapper| false)
175     }
176
177     pub fn generate_until<F>(&mut self, should_halt: F) -> String
178     where F: 'static + Send + Sync + Fn(&WordWrapper) -> bool {
179         let mut threads = Vec::with_capacity(self.thread_number);
180         let thread_number = self.thread_number;
181         let arced_should_halt = Arc::new(should_halt);
182         for in 0..thread_number {
183             let mut missed_pops = 0i32;
184             let queue = self.task_queue.clone();
185             let max_len = self.max_len.clone();
186             let should_halt = arced_should_halt.clone();
187             let local_templates = self.template_vecs.clone();
188             let new_thread = thread::spawn(move || {
189                 loop {
190                     let current_word: WordWrapper;
191                     {
192                         let mut queue_lock =
193                             ↪ queue.lock().expect(&format!("Failed at line {}", line!()));

```

```

189         match queue_lock.pop() {
190             Some(task) => {
191                 missed_pops = 0;
192                 match task {
193                     Task::ProcessWord(word) =>
194                         ↪ current_word = word,
195                         Task::Halt => return None,
196                     }
197                 None => {
198                     //println!("miss {}", missed_pops);
199                     if missed_pops > 100_000 {
200                         return None;
201                     } else {
202                         missed_pops += 1;
203                     }
204                     //thread::sleep_ms(5);
205                     continue;
206                 }
207             }
208             }
209         let children: Vec<WordWrapper> =
210         ↪ current_word.spawn_children()
211             .into_iter()
212             .filter(|wr|
213                 ↪ is_template_vector_free_partial_check(wrd, &local_templates))
214                 .collect();
215         let mut queue_lock =
216         ↪ queue.lock().expect(&format!("Failed at line {}", line!()));
217         for child in children.iter() {
218             if should_halt(child) {
219                 for _ in 1..thread_number {
220                     queue_lock.push(Task::Halt);
221                 }
222                 return Some(word_to_string(child));
223             }
224             for child in children.into_iter() {
225                 queue_lock.push(Task::ProcessWord(child));
226             }
227         }
228         threads.push(new_thread);
229         let mut result: Option<String> = None;
230         for thread in threads {
231             if let Some(res) = thread.join().expect("Could not join
232             ↪ thread") {
233                 result = Some(res);
234             }
235         }
236         result.unwrap_or("").to_owned()
237     }
238 }
239
240 fn calculate_parikh_chain(word: &Word) -> Vec<[usize; cfg::CARDINALITY]> {
241     let mut result = Vec::with_capacity(word.len() + 1);
242     result.push([0; cfg::CARDINALITY]);
243     for i in 0..word.len() {
244         let mut new_elem = result[i].clone();
245         new_elem[word[i] as usize] += 1;
246         result.push(new_elem);
247     }
248     result
249 }
250
251 pub fn calculate_parikh_vector(word: &Word) -> [usize; cfg::CARDINALITY] {
252     let mut result = [0; cfg::CARDINALITY];
253     for letter in word.iter() {
254         result[*letter as usize] += 1;
255     }
256     result
257 }
258
259 fn parikh_diff(p1: &[usize; cfg::CARDINALITY], p2: &[usize;
260     ↪ cfg::CARDINALITY]) -> [isize; cfg::CARDINALITY] {

```

```

260     let mut result = [0; cfg::CARDINALITY];
261     for i in 0..p1.len() {
262         result[i] = (p2[i] - p1[i]) as isize;
263     }
264     result
265 }
266
267 pub fn is_aa2f_partial_check(word: &Word) -> bool {
268     if word.len() < 4 {
269         return true;
270     }
271     let l = word.len();
272     let parikh_chain = calculate_parikh_chain(word);
273     for i in 2..(l / 2 + 1) {
274         let diff2 = parikh_diff(&parikh_chain[l - i], &parikh_chain[l]);
275         let diff1 = parikh_diff(&parikh_chain[l - 2 * i], &parikh_chain[l
    ↪ - i]);
276         if diff1 == diff2 {
277             return false;
278         }
279     }
280     true
281 }
282
283 pub fn is_aa2f_full_check(word: &Word) -> bool {
284     let l = word.len();
285     for part_len in 1..(l + 1) {
286         for window in word.windows(part_len) {
287             if !is_aa2f_partial_check(&window.to_vec()) {
288                 return false;
289             }
290         }
291     }
292     true
293 }
294
295 pub fn is_template_vector_free_full_check(word: &Word, templates:
    ↪ &Vec<[isize; cfg::CARDINALITY]>) -> bool {
296     let l = word.len();
297     for part_len in 1..(l + 1) {
298         for window in word.windows(part_len) {
299             if !is_template_vector_free_partial_check(&window.to_vec(),
    ↪ templates) {
300                 return false;
301             }
302         }
303     }
304     true
305 }
306
307 pub fn is_template_vector_free_partial_check(word: &Word, templates:
    ↪ &Vec<[isize; cfg::CARDINALITY]>) -> bool {
309     let l = word.len() as isize;
310     let parikh_chain = calculate_parikh_chain(word);
311     for templ_vec in templates.iter() {
312         let norm = templ_vec.iter()
313             .fold(0, |acc, &x| acc + x);
314         let mut i: isize = 1;
315         while (i * 2) as isize + norm <= l as isize {
316             let diff1 = parikh_diff(&parikh_chain[(l - 2 * i - norm) as
    ↪ usize], &parikh_chain[(l - i) as usize]);
317             let diff2 = parikh_diff(&parikh_chain[(l - i) as usize],
    ↪ &parikh_chain[l as usize]);
318             //println!("Diffs are {:?} {:?}", diff1, diff2);
319             let mut are_eq = true;
320             for j in 0..cfg::CARDINALITY {
321                 if diff1[j] - diff2[j] != templ_vec[j] {
322                     are_eq = false;
323                 }
324             }
325             if are_eq {
326                 return false;
327             }
328             i += 1;
329         }

```

```

330     }
331     true
332 }
333
334 pub fn word_to_string(word: &Word) -> String {
335     let utf: Vec<u8> = word.iter().map(|&x| x + 97).collect();
336     String::from_utf8(utf).expect(&*format!("Failed at line {}", line!()))
337 }
338
339 pub fn string_to_word(in_str: &str) -> Word {
340     in_str.as_bytes().iter().map(|x| x - 97).collect()
341 }
342
343 pub fn generate_backtrack(word: &Word) -> Vec<Word> {
344     let mut result = Vec::with_capacity(word.len() - 1);
345     for i in (1..word.len()).rev() {
346         result.push(word[0..i].to_vec());
347     }
348     result
349 }

```

## 6. Contents of the src/cfg.rs File (Classifier)

```

1 pub const CARDINALITY: usize = 3;
2 pub const HEAP_CAPACITY: usize = 1_000_000;

```

## 7. The Thread Communication Methods Test

```

1 extern crate time;
2
3 use time::*;
4 use std::sync::{Arc, Mutex};
5 use std::sync::mpsc::*;
6 use std::thread::*;
7 use std::cmp::min;
8
9 const NUM_COUNT: i64 = 5_000_000i64;
10 const SPAWNED_THREAD_COUNT: usize = 4;
11 const BULK_SIZE: usize = 10;
12
13 fn collatz_length(mut num: i64) -> i64 {
14     if num <= 0 {
15         return 0;
16     }
17     let mut result = 1i64;
18     while num != 1 {
19         result += 1;
20         num = match num % 2 {
21             0 => num / 2,
22             1 => 3 * num + 1,
23             _ => 1,
24         }
25     }
26     result
27 }
28
29 fn test_serial() {
30     let numbers_to_process: Vec<i64> = (1..NUM_COUNT+1).collect();
31     let mut sum = 0i64;
32     let start: u64 = precise_time_ns();
33     for num in numbers_to_process.iter() {
34         sum += collatz_length(*num);
35     }

```

```

36     let end: u64 = precise_time_ns();
37     println!("Sum is: {:?}", sum); //to try to avoid compiler optimizing
    ↪ it away
38     println!("Took {} ns to compute in serial manner", end - start);
39 }
40
41 fn test_shared_memory() {
42     let numbers_to_process: Arc<Mutex<Vec<i64>>> =
    ↪ Arc::new(Mutex::new((1..NUM_COUNT+1).collect()));
43     let mut sum = 0i64;
44     let start: u64 = precise_time_ns();
45     let mut threads: Vec<JoinHandle<_>> =
    ↪ Vec::with_capacity(SPAWNED_THREAD_COUNT);
46     for _ in 0..SPAWNED_THREAD_COUNT {
47         let nums = numbers_to_process.clone();
48         let handle = spawn(move || {
49             let mut local_sum = 0i64;
50             let mut task_queue: Vec<i64> = Vec::with_capacity(BULK_SIZE);
51             loop {
52                 if task_queue.len() > 0 {
53                     ↪ let num = task_queue.pop().expect("Pop result
    shouldn't be empty at {}");
54                     local_sum += collatz_length(num);
55                 } else {
56                     ↪ let mut lock = nums.lock().expect("Mutex was
    poisoned");
57                     if lock.len() == 0 {
58                         ↪ return local_sum;
59                     } else {
60                         for _ in 0..min(lock.len(), BULK_SIZE) {
61                             ↪ task_queue.push(lock.pop().unwrap());
62                         }
63                     }
64                 }
65             }
66         });
67         threads.push(handle);
68     }
69 }
70
71 for join_handle in threads {
72     sum += join_handle.join().unwrap();
73 }
74 let end: u64 = precise_time_ns();
75 println!("Sum is: {:?}", sum);
76 println!("Took {} ns to compute in parallel using shared memory with
    ↪ bulk size of {}", end - start, BULK_SIZE);
77 }
78
79 fn test_channels() {
80     let mut numbers_to_process: Vec<i64> = (1..NUM_COUNT+1).collect();
81     let mut sum = 0i64;
82     let mut threads: Vec<JoinHandle<_>> =
    ↪ Vec::with_capacity(SPAWNED_THREAD_COUNT);
83     let start: u64 = precise_time_ns();
84     let (main_tx, main_rx): (Sender<Option<Vec<i64>>>, i64>,
    ↪ Receiver<Sender<Option<Vec<i64>>>, i64>>) = channel();
85     for _ in 0..SPAWNED_THREAD_COUNT {
86         let tx = main_tx.clone();
87         let handle = spawn(move || {
88             ↪ let (local_tx, local_rx): (Sender<Option<Vec<i64>>>,
    Receiver<Option<Vec<i64>>>) = channel();
89             ↪ tx.send( (local_tx.clone(), 0) );
90             while let Ok(new_tasks_opt) = local_rx.recv() {
91                 if let Some(new_tasks) = new_tasks_opt {
92                     ↪ let mut local_sum = 0i64;
93                     for i in new_tasks.iter() {
94                         ↪ local_sum += collatz_length(*i);
95                     }
96                     ↪ tx.send( (local_tx.clone(), local_sum) );
97                 } else {
98                     ↪ return;
99                 }
100             }

```

```

101         });
102     });
103     threads.push(handle);
104 }
105 drop(main_tx);
106 while let Ok( (tx, collatz_result) ) = main_rx.recv() {
107     sum += collatz_result;
108     if numbers_to_process.len() == 0 {
109         tx.send(None);
110     } else {
111         let mut data_bulk: Vec<i64> = Vec::with_capacity(BULK_SIZE);
112         for _ in 0..min(BULK_SIZE, numbers_to_process.len()) {
113             data_bulk.push(numbers_to_process.pop().unwrap());
114         }
115         tx.send(Some(data_bulk));
116     }
117 }
118 }
119 for join_handle in threads {
120     join_handle.join();
121 }
122 }
123
124 let end: u64 = precise_time_ns();
125 println!("Sum is: {:?}" , sum);
126 println!("Took {} ns to compute in parallel using channels with bulk
↪ size of {}", end - start, BULK_SIZE);
127 }
128
129 fn main() {
130     test_serial();
131     test_shared_memory();
132     test_channels();
133 }

```

## 8. The Results of Thread Communication Testing in CSV Format

Task Size,Threads Spawned,Bulk Size,Serial,Shared Memory,Channels

```

5 000 000, 4,1,3424578206 ns,1778776155 ns,5224623987 ns
5 000 000, 4,1,3414758847 ns,1772402450 ns,5220775848 ns
5 000 000, 4,1,3433412774 ns,1774028450 ns,5226536009 ns,
5 000 000, 4,2,3441499936 ns,1452454907 ns,3634465568 ns
5 000 000, 4,2,3429163043 ns,1382693992 ns,3447613498 ns
5 000 000, 4,2,3419446758 ns,1384678747 ns,3423935213 ns
5 000 000, 4,3,3422137868 ns,1299928651 ns,2759338231 ns
5 000 000, 4,3,3416833290 ns,1297326338 ns,2756429815 ns
5 000 000, 4,3,3420646624 ns,1293847216 ns,2798645566 ns
5 000 000, 4,4,3420009433 ns,1262555202 ns,2435675965 ns
5 000 000, 4,4,3430368711 ns,1259968506 ns,2425848128 ns
5 000 000, 4,4,3429328588 ns,1267493439 ns,2443529312 ns

```

5 000 000, 4,5,3467159851 ns,1312614481 ns,2314144099 ns  
 5 000 000, 4,5,3424305570 ns,1248111293 ns,2250099617 ns  
 5 000 000, 4,5,3434055320 ns,1245624102 ns,2252204849 ns  
 5 000 000, 4,10,3423886577 ns,1223353174 ns,1924358000 ns  
 5 000 000, 4,10,3437502314 ns,1220080203 ns,1926680092 ns  
 5 000 000, 4,10,3421918331 ns,1220700884 ns,1928178921 ns  
 5 000 000, 4,50,3428462489 ns,1211860069 ns,1568163250 ns  
 5 000 000, 4,50,3411500601 ns,1208260024 ns,1569898571 ns  
 5 000 000, 4,50,3434107974 ns,1206328369 ns,1571151091 ns  
 5 000 000, 4,100,3421820164 ns,1213313385 ns,1438375553 ns  
 5 000 000, 4,100,3456933992 ns,1216621160 ns,1438201530 ns  
 5 000 000, 4,100,3433728247 ns,1206304273 ns,1439515627 ns  
 5 000 000, 4,1000,3570326043 ns,1314629578 ns,1264423942 ns  
 5 000 000, 4,1000,3557772737 ns,1309710082 ns,1264052692 ns  
 5 000 000, 4,1000,3551442316 ns,1309506163 ns,1259236717 ns  
 5 000 000, 4,10000,3542614888 ns,1304465296 ns,1242568436 ns  
 5 000 000, 4,10000,3541726032 ns,1307653933 ns,1238440522 ns  
 5 000 000, 4,10000,3540885366 ns,1310332103 ns,1240470344 ns

## 9. Almost Abelian Square Free Word of Length 15796

abaccbbbccbaacccaaaccbbbaabbbcbabbcccacbcccaaaccbbbacccbbaaacaaabaaaccbba  
 bbbcbbaaccbcaaabbcccbbbccabbcccccaccbcaaabaaccbbbaacbaaabbcbbbaaccbcccacbb  
 aaabbbcbabbabbccaaabbbbaacaaabaaaccbcaabaaacaabcbabbcccaaacbaaccbcccaccc  
 bbbaccaabbcbabbbaacabcaccbbbaabccaaaccabccaaabaaacaaabbbbaaccbbaabbbaac  
 cbbbcccabccaabaaacaabbbbaacccaaabbcaabbbcbabbcccbbbccaaabbbbaacaabaaaccbabb  
 ccbaacccaaabaaacaabbbbaaccbbaabbbaccbbbbcbbaccbbbaacaabaaaccbcbabbccaccc  
 bcccaaaccbbabbcccbbbccaabcbbbbaabbbccabccaaaccabbcccbbbacaabaaccbbbaacbbaa  
 abbbcbabbabbccacbabbbcbbaaacaabbccaaacccaabaaacaabbccaaabbbaccbccaabccaa  
 abaacbbabbbbcccbbaaabcacccbcccaabbbcccbabbbbcbacaabbbcccaaaccbbaabbbaccbbb

cccabccaaabaacbbbcacbccaaacbbacbccacccbbacbbcccaaaabacbabbbcccbaaabbbcb  
babbbccabbcccbbaaacbccacbbcccbbaaacaabaaacccbbbabbbcbbaaabbcaabbbcbba  
bbcccbaccaabbbcbabbccaaacbbaaabbbaacbbaaacaabacccaaabbccacccbbabbcb  
acaaabaacccbbaaabbbcbabbccabbccacbcccaaaccbabbbcccbbaaabbcbabbccca  
bbcbbaaacccaaabaacaabbbcabbbcccaaaccbccaaabbbbaaacccaaabaacaabbbacbbaaac  
aaabaaacccbccaaabbbcccbabbcbbaaacabaccbbbaaacaabaaacccabcccbbaacbcccaa  
acbbaaabbbaacbbabcccbbaaccabcccbabbcbbaaacabaccbbabcbbaaacaabaccab  
bcccbacaaabbbcbacccabbcccbbbccabbccacccbcccaaaccbbccacccbccaaabaacaabb  
cbaaccbbabbcbbaacaabbbcccaaaccbcaccbbcccaabbbbaabcbabbcccaaabbbbaaaca  
baaccabbcbbaaacaabaaccaabbbcccbabbcbbaaacbccabbccacccbcccaaabaacabbbaa  
acccaaabaacaabbbcaabbbcacbaacaabaacbbabbcbbaacabbcccbacccaaabaacaabbc  
aabbbaabbbcccaaabaccbbaaabbbcbabbcccbbaaacbbcaaaccbbbaabbbacbbaaacaabaa  
accbbacbbbbccacccbcccaaabbccacccbabbbcbacaaabbbacbbccaaaccabbcccbbbccaaa  
bbcccbabbcbbaaabbcccbbaaabbbaacaabaaacbbaccbbbaaabbbcbabbccacbb  
aaacaabbbccacccbbbaaacccaaabaacaabbbcaabbbcccaaabaacaabbbcbabbccacccb  
cccaabaccbbbbcccbabbcbcaaaccbbabbcbbaacaabaaccaaaaccbbbaaacaabcccbacbbab  
cbbaacccaaabbcaabbbbaacaabaaacbbaacccaaaccbaaabbcaaacccbacbbabcbbaaacbbca  
ccbbbbcccaaaccbbccacccbbacbbbbccbbaccaabbbcccbabbcbbaaacabccaaabaaca  
aabbbaacccaaaccbaaccbbbccabbbaacccaaaccbaaccbbccacccbbccaaaccbbccacbbcc  
bbaaccbaabaacaabbbccaabbbcbabbcccbbaacccaaaccbccacccbbccabbbaacabbcccb  
abbbcbbaabbbcaabbbccaabcaaacccbbccaaabbbaacbbaaacaabaaacccbccacccbbabb  
cbacaaabbcaabbbcbabbcccbbaaacccbccabbccacccbccaaabaacabbbaacbbccacbbaac  
babbbcbacbccaaabbbbaacaabbbcbbaaacaabaaacbbaacccbcabaaacaabbbacbbaacccbb  
bccacccbccaaaccbbbaabbbcbabbbbccaaabbbcaabbbbaacaabaccabbbaaacabcbbaaa  
bbcccbacbbabbcbbaaabbcccbbaaabbacbbcccbbaaccaabbbcccbacaaabbbcb  
bbaacaabaaacccabbcbbaaacaabbbcccaaaccbccacbbcccaaaccbbabbcbbaacccaaa  
baaacaabbbcbbaaccbaaabbcbabbcccbbaaacccbccacbbabbcccbbaaabbcbba  
bbccacccbbabbcbbaaabbcccbbaaacaabaccbbcccbbaaacccbccacccbbabbcbba  
aabccacbbabbccacccbccaaabaacbbccaaacbaaabbbaacbbbcacbaaacccabccbbbcabb  
aaabcccbacbbabbcbbaaabbcaaacbbbaacbbabbcccbacccaaabbcbbaaabbcccaaab  
aaacaabbbcbabbcccaaabbbbaacaabbbcccaaaccbaaccbbccacbbbaaacabaaacbbba

cbabbbcccabbbbaaacbaccbaabbbcccbabbabbcbbaaabbcaabbbccaaabaaacabbbcccaaac  
bbaaabbbbaacbccaaabbbbaacaaabaaacbaacccaaabaaacaaabbbcccacccbccaaabbbbaabcc  
caaabaaacaaabbbbaaacbbbaacbaaaabbbcbabbabbccabbcccbbaacccbbaabbbacbbbcca  
bbcccacccbcccaaabaaacaaabbbbaacbbaaacccabcccbbabbcbabbbaaabcccabbcbabbccab  
ccbbbabccaaacccbccacccbbacbccaaacbbbaabbbbaacbbccabbcbabbbaacccbcccaaacb  
bbcacbccaaabaaacaaabbbbaaacccabbbcbacccaaacbbabbcbabbbaaabbaabbbcbabb  
bccbbbaaabbbcbabbcccbbaaacbbcccabcbabbabbcccaaacbbacaabacccbabbcbabbbaa  
acbccacccbbbccaaabbbcccbabbabbcbbaaacbbccacbbbcccaaaccbccacccbbacbbbcc  
accbbccaaabaaacaaabbbcaabbbbaabbbcaaacbbabbcbbaabbbccaaacbbbaabbbcbabb  
cacccbbabbcbabbbaacccaaabbbbaacccbccacccbbabbcaaacbbacbbbaacaaabaacbccaaa  
cbabbcacccbbabbcbabbbaacbbbaacaaabaacccaaabbbbaacbbbccbbbaacccaaabaacbbaaaca  
aabaacccaabcccbbaacccaaabacbabbbccacccabaaacaaabbbbaacbbbccaaabbbbaacccacca  
abbbacbbbccbbbaacccbccaaacccbbabcbbaaacccaaabacbbbaacaaabaacccaabcccbabcb  
aacaabaacccbcacccbbbccaaabbbacbbbccacccbccaaabcacccbbbaacccaaabaaacaaabbc  
ccaaacccbccaaabbbccbbbcabbbaabbbccabbbbccaaacccbccacccbbacbbbccbbacccaaacbb  
bbabbcccbbbccaabbbcbbaaacabaaacccabcccbbbaacccbbabbcccbbaaacccbbbccacccacca  
abaaacaaabbbcbbaacaaabbbccaaabaaacabbbcbbaaabcccbacbbbabcbbaaabbbccaaab  
cbabccaaacccabbcccbbbccabbbaaacccabccaabbbcbabbabbccaaabbbccbbbcacccbcccaab  
bccbbbccabbbaaacabacccbbbabccaaacccbccacccbbcabcccacccbcccaaabaaacabcbba  
aabbcccaaabbbbaacbbbccbbacccaabbbccbbabbcbabbbaacccbbbaabbbcbbaacaaabaaccc  
abbccbbbaacaaabbbacbbbccabbccacbcccaaabaacccbccacbbbaabbbbaacaaabaacccbaac  
aaabbbbaacbbbaabbbbaabbbcaabbbcbabbabbcccbbaacbaaabbcbbaacaaabaacccbabbb  
aacbcacccbbabbcbbaacaaabbbcbabbabbcccbbaaacbbbccbbbaacccaaacbbbabbbcbabb  
aacccbccaaabaaacaaabbbacbbbaacbbabbcbbacccbccaaabbbcacbaacccbcacccbbbccaa  
abbbacbbbccbbacccaabaaacaaabbbbaacbbbccbbabbcbbaacaaabaacccaaabbbbaacaa  
aabaacccbaacccaaabaacaaabbbcacccbcccaaaccbbabccaabbbbaacbbacccbbbaacccaaaba  
aacaabbbcabacccbbacaabbbbaacbbbccbbacccbaacaaabaacccbbbccacccbcccaaaccbb  
babbaabbbcbabbabbccabbbaacaaabbbccaaabbbcabcccbacaabaaacccbccacbbabbbc  
ccbacbbbaaacccbccacccbbbccabbcccaabbbcbacccbccaaabaacaaabbbacbbbccabbbaaac  
cabccbbabccacccbccaaabaacaaabccacccbbbaabbbacaabaaacccbbabbcbbaaacbaacaaa  
bbbacccbccaaabbcbaacbbbccbbabbcbbaacbaaabbcbbaacbbacbbbccacccbcccaabb

cccacbbabbbcaabaacbbbabbbcbbaacccbcaaaabbbbaacaabaacbbabbbcbbaacccbcccac  
bbbaaacbaacccbcccacccbcbcccaabbcccbabbcbbaacccbcbccacccbcccaabaacbb  
baaabcaaacbaacccbcccacccbcbcccaaacbccacccbabbabbbcbacaabaacccaaabbcccacbbb  
ccaaabaacccabccaaabaaacaabbbaacbbabcaabaacccbcaabbaabcccabbcbabbabbaaba  
aacaabcbbaaabbbcccaaacccbcaacbbacbbbabcbabaaacabccaabbbcacbaacccbcccacbbb  
ccabbcccaabbbcbbaaccaaabacbbbbbccbbbaabbbcbabbcccbcbccaaabaaacabbbaaabcc  
aabaacccabbcbcccacbbbaaacbacbbbaabbbcbbaacbbaccaabbbcbabbcbcccacccbccaabc  
abbccbacccaabaaacabbbaabbbccaabaccabbccbbbccabbbaaabccaaabbbbaacaabaa  
acccaaabbccacccbbaaacaabaaaccaaabccaaabbbccacccbccaabbbcabaaaccaaabbcccb  
baaaccaabaacaabbcabbbaaacbbcaabbbccabbcccacbccaaacbbbaabbbcaaacbbccaabbbc  
cbbbabbbcbacaabbbcbbaacccbcaaaabbbacaaabaaacccbcccacbbbcccaaacccbcccacbbb  
acbbbaacabbccacccbcaaacbaaabbbcaabbbcccacbbbaabbbcbabbccabccbbcbcaaacba  
aabbbaabacbbbbbcbabbcbbaabbbbaacccbbaabbbcbbaaacaabaaacbbcaabbbccab  
ccbbabbbcbbaacbacbbbbbcbacccaabbcacbbbaacbabbbcbbaacbbbaabcccaabaaacaab  
bbaaaccaabbbbaabbbcbabbbbbcbbaaacbaacccaacbaaabbbbaacbbbcccaabbbcbabbcc  
cbacbbbaacbbcaabacabbbaacbbbbbcbacccaabbbcccacbbacaaabaacbbcaabbbbaabbc  
aaacccbcbcccacbbabbcbbaacabbcbbaacccaabaaacabbbaabbbccabcbbaaaccaabccaaab  
aaacaabbbbaacccaacbbbaabbbacbbbbcabaacccaabbcabbbaaacaabacccaabbbcccaccc  
bcaaaccaabbbbaacaabaaacbbbaacccaabbbcaaacccbabbcbcaabaacccaabbbcaabbbbaa  
caabaaacbbbaacccaabbbcabaaacaabbbcabbbbaabbbccbabbbcbbaacbbbaabbaaacbbab  
bbccbbbaacccaabbbcabaaacaabbbcaabbbbaabbbcacbbbaabbbcbbaacccbcaaacbaacbbcca  
cbbbbcabbccaaacbbbaabbbbaacbbbaacbbbbbcbbaacccaabbaaacaabbbcaabbbcbbaaa  
bbcccaabcaabbbbaacbbbbbcaabbbbaabbbcaabbbcbabbabbcccaabbbbaabbbcbbaacccabaa  
cbbbbbcbbaacbbacccaabbaaacabcbbaabbbcccaabaacaabbbcbabbabbccacbaaabbcaccc  
bbbabbbcccacccbcccaaacbbacccbbaabbbbaacaabaacbbbaabbbbaacbbbbcabaacbbcccbb  
accbaabaacaabbbcabaaacbbbaabbbcbabbabbccabbcccacccbcccaabbbccacbbbabbbcc  
aaabbbaccbaabccaaabaaacabcccbbbaabccaaacccbccacbbbbbcbbaabbbcaaaccaabccaaa  
baaacaabbbbaacccaabbaaaabbbccacbbbaacccaacbbccacbbbaacbbabbcbbaabbbbaacc  
babbbccbbbaacbaaacbbcaaaacaabbbbaaacbbbaacccbbaabbbcaabbbcbabbabbccbbbaa  
abbbcbbaacbbcacbbbaabaaacaabcbbaacccbcccacccbbaaacaabaaacbbbabccacbbcaaac  
cbbaaabbbaabbbcbabbabcccbaaacbbccacbbbcccaabbaaacabbbbaabbbccacbbbcccaaa

cccbccccbbcbabbbaaacccbccaccbbccaabbcbabbccbbccaacccaabbbcbbaaab  
bbaacbcaccbbbaaabbbcbabbccabccbbabbcbbaacaaabccaccbbccaabcbabbcc  
ccbbbaaccbbccacbaaccbbaaacbccbbbaabbaaaacbaaabcbabbccbbbaaabbbcc  
baaacaabbbccaccbcaaaccaabbbcbbaacaaabaacbbabbccaabbbaccbaacccaacbbba  
aacaabaaacccaabbbbaaacabbbcbbaacaaabaccaabbccaccbbbaaabbbccaaacbccacc  
bbacbbbbccbbaccbaaacbbbaaababccaaabbbcbabbccbbccaacccbccaccbbabcbbaa  
acaabaaacccaabbbcaabbbaccbbbaabbbbaaabcaaacccccaabbbccbabbbcbbaaabcccaa  
bbbccbbabbcbbaacbbabbcccaabbbbaaabcaaacccbaaabbbbaaccbbccbbbaacccaac  
baaccbbbaabbbacaabaaacbbabbcbbaaacbbaccbbabbcbbaaabbbccaaacbbbaaa  
bbbaacbbbaacaaabaaacccaabbbbaacabaaacccabbbccaccbcaabaaacccbbbaabbbcbabb  
bccabaaacccaabbcaccbbbcabbccaccbccaacccbbbaacaaabaaacccaabbbcbabccaaacca  
abcbabccbbbaabbbbaabbbccabaaacaaabccbbbaaacaabccaaabaaacaaabbbbaacccaab  
bbbaabbbcaabbbcbabbcccbacbbbaabbbbaaccbaaccaabbbbaacaaabaaacbaaccbbabbbcc  
cbaacccaabacaaabbbbaaacbbabbcbaccbbbaaacabccbbaccbbabbcbbaaabbbccca  
aabbbbaacbbbaacaaabaccabbcbacaabbbcbabbcccbabbcbbaaabccaaacccbccaccbb  
baaacccbabcbbaacbccaaacbaaabbaaacbaaccbbccaccbbccaacccbccaccbbabbcc  
bbbaaacbaacaaabbbaccbbcccbaccbbbaaccbaaccaabaaacaaabbbaccbbccaabbbbaabbc  
ccaaabbbbaacaaabaaacccaabbbbaacccaacccbbbaaccbbbbccaccbbccaacccbaaccbb  
bacbaacccaacbaaabbbcaabaccbbbccabbbaaacabbbccaccbbccaabccaaacccbccaccbb  
babccaccbbccaabaacbbaccbbcccbaccbbbaaabbcaccbbabbcbbaaacabaaacccaabbbbaa  
acabbccbbabbcccaabaccbbcccbaccbbbaacccaabaaacaaabbbbaacbccaaabaacbbabbcc  
caaabbcbabbbbccabbcccbaccbaabaacabbbbaaabccabccbbbaaabbbcbabbccaccbbbc  
caaabbccbbabbccaccbbccaacbbacbaaacbbabbcbbaaabbbcccbaccbaabaacabbbbaaa  
bbccaacccbbcccbaccbbccaabaacccbccaccbbbaabbbaccbbcccbaccbaabaacbbba  
bbcbbaaabcaaacccbaaabbbccaccbbcabbbcccbacaabbbcbabbccbbbaaabbbcbbaaac  
aabbcabaaacccaabcccbbaaabbbcbabbccabbccabcaabbbcbbaaccbbccaccbbcccaabb  
bcbabbccbbccaabcaaacbbabbccbbbaaabbbcbabbccabccbbabbcbbaaac  
baaccbbccaccbbbaacbccaaabaacabcbabbccabbbaaabbaaacccbbbaacaaabbbcbbaa  
acaabacbbcccaabaacaaabcccbaccbbacbaaabbbccacbaacaaabbaacbbccaabcbabbcc  
cbbaccbaabbbcaaacccbccaccbbaccbbbaaabccaccbbccaabaacabcbbaacaaabaaccc  
caabccaabaacaaabbbcccbaccbbcabbbbaaabcaaacbbacaabbbcbabbccaccbbccaaba

caaabbbaacbccaaacbbaaabbbacbbbbbccacbbabbbcbbaaacccbbbcaabbbbaacaaabaaaccc  
bcccacbabbbcccacccbccaaabccaaacccbccacccbbbcccaabcbbaaabccccacccbcccaabaaacc  
cbccaabbbbaabccacccbbaaabbbcbbbbabbccabcccbbabbbccaabcccccbbccaabbbcccbbab  
cbbaacabaacccbbccabaaacccbccacccbbacbbbbbccbbaccaabbcbbbbabbcccbbaaacccaa  
bcbbaacaaabaaacccbccacccbbbcccaabccaaabbbcbacbbabbbcbbaaacaabaaacccbbbaaac  
abcccbbabbbcbbaaaabbbcbbbbabbccacccbbbccaaacccbccaaabaacccbaabbbbaabbbbbb  
abbcccbbbbcaabcbbaaabccacccbbcbabcccacccbccaaacccbbaccaaabcbbbbacaaab  
acccaaabccacccbbbcaabbbccbbbabbbcbbbbbaaabcaabbbcccbabbcccaabbbbaacabcca  
aabaacaaabbbcbacccaaacccbccabbcbbbbacaaabaaacccbbbaaabbaabbbcccbacbaabbb  
cbbaacccbbbcccacccbcaaaccbbacaaabaccaabbbcccbacaabaacbbbabbaaabaacccbbba  
caaabaacbaacccaaabbbccacccbcaaaccbaabbbacbbbbbccacccbcccaabccaaacccbaabccbb  
acccaaacbaaabbbbabbbcccbbaaccabbcccbbbbccabbccacccbcccaaaccbbabcbbaacca  
aacbaacccbccacccbbbaabbbcbacbccaaabaacbbbbcabbbaaacaabaaacccbacbbbaa  
acaabbbccaaacccbccacccbbacbbbccaaacbccacccbaaacaabcbbbbbaaccabcccbbbabbbc  
bbbaaabcccbbabbaaabaacbbbaaacccbbacbbbbbccbbaccaaacbaaabbbcccaabbbacaaa  
baaacbaacabbbcccbacccaaacbaacccbccacccbbbcccaabbbcccbacccaaabaacaaabbb  
cbaacaabbbcccabcaaaccabcccbbacbbbabbbcbbaacaabccbbabccaaabbbbaacaabaaa  
cccabccaaabaacaaabbbcccbbabccacccbccaaabaacccbaabbbbaacaaabaaacccbaacccaaac  
cbaaabbbcaaacccbccacccbbacbccaaabbbcccbabbcbacaaabaccbccacbaacaabaacbc  
cacbbabccaaabcaabbbcbbbbabbcccbaccaabbcccacbbbbbccbbacaaabaacccaaabcccbba  
bccaaacccbccacccbbacbbbbbccbbaccaabcccbbbcabaaacccbaaacaabaacccaaacbabbc  
aabaacccbccacccbbccabbccacccbccaaacccbbcccacccbccaaabaacccbacbbbaabbbcb  
aacccabbccbbbbbccabbccacccbccaaacccbbcccacccbccaaabbbbaacbbbbbccacccbccaaabbb  
cbbbabbbcccbbaaccabcccbbccaabaacccbbbaabbbacbbbbbccbbbaaacabaaacccaaabcccb  
bbabbccaabbbccacccbccaaabccaaacbbccacccbccaaabaacaaabcccbbbbccaabbbaacabc  
caabaacaabbbcbbbbabbcccbbaacccbccacccbbccaabcbbaaacccabccaabaacbccaaabb  
bacbbaaacaabaaacccbccaaabaacaabbbccacccbccabbbaacbbbaaacabaccbbbabcbbaaab  
bbccabbbaaacaabaaacccaaabbbccabaacaabbbbaabbbccbbbabbbcbbaacbabbbccacc  
cbccaaacccbbacaabbbccbbbabcbbaaacbbcccbaabaacaaabbbcbbbbabbccabbcbba  
aacbcabbccacccbccaaacccbbbaabbbacbbbbbccacccccaabcbbaaacbbacaabaacccbac  
caabbbccacbbbbbccbbbaaaccaabccaaabbbbabbbccbbbaacbbbaabcccaabaacbbbaacaa

abaaacccaaabccccacccbccaaacbbacabaaacccaaabccccbbccaabbbbaaacccbccacccbbb  
ccabccccacccbccaaacccbbbccccacbbaaabccaaacccbcaabbcccaaabaacaaabbcaabbbbaaab  
bcaaacccbbbcaaaabcbbbbabbccccbacbbbaacccabbccccbbcabccccacccbccaaacccbbbccccac  
bbaaabbbbaacccbcabaaacaaabbbcbbbbabbccccabaacaabbbbaacccabbccccbbabbcbbbbbaacb  
babbbccccbbbaaabccccabaaacaaabbbcbabbccccbbaacccbccaaacccbbbabccaaaabaacaaabbba  
aacccbbbccabaaacccbbbaabbbcbabbccccbbbaacccabbccccbbabbccaabaaacccbccaaabaa  
acabccaabbbacbbbccccbacbbbaacccaaacbaacccbccacccbbbaccabacabbbacccbbbccabb  
baaacbbabccccbbaacccaaabacabbccccacbccaaacbbbacaaabaacccbaacccaaacbabbbcbbbb  
aaacaaabaacccbbbaacaaabaacbbabbbcbbaacaabbbcacccbcabbbbaacaaabaacccaaabccac  
ccbccabbbcbbaacabbccccacbacaaabbbcbbbbabbccabbccccacccbcccaaabaacccbbbcccbabbb  
cbaaabbbccccacbbbaaabbbbaacccaaabaacacbbbaaabbbccaabbbbaacabcbbbbabbcbccbbbaa  
ccbaaabbbcbbaacccbaabaaacccbccacccbbacbbbccccacbbbaaabbbcbbaacaaabbbcbbbbabb  
bccccbbbaacabbccccbbccabbccccacccbcabbbbaacabccccbbacbbbabcbbbbbaaabbbccbaacaa  
abbccabcaaaccabccccbbccaabcbbbbabbbccaaacccbbbccaabaacccbccacccbbbcccaaaccbb  
cccacccbbacbbbbbccccacccbcccaaabbbcbabbbccccbbbaacccbccabaacaabccccbbbabcbacaa  
abaacccabbbcacccbcccaaaccbbbaacaaabacccaaacccbbacbbbaacbbbaacccabccccbbabcccc  
aaabbbcbbbbabbccabccccbbbaacbbbbbccccacccbccaaabccaaacccbbbccccacbabbbccccbbbaa  
ccbaacaaabbbbaaacccbbbabbaabbbcbbbbabccccbbccaacbbbaaabbbccabaacccbccacccbb  
babbbcbbbbbaacabccccbbbaaacbbbaacaaabbbbaaacccbbcbabaaacccbccacccbaaacaabbbccaaa  
baacaabbbbaaabccccbacaaabbbcbabbccccbaacccaaabcbbbbbaaabcaaacbaacccbccacbaac  
cbbbbcaaabbbcbbbbabbccccbaacccabcbbaaabcaaacbaacccbccacccbbbabccacccbccabaac  
aabbbccaaabbbbaacaaabaacccbccacccbabbbccccacccbccaaabaacccbccaaacccbbacbaaabbb  
bacbbbbbccbbacccaaacccbbbaaabbbccaaacccbccacccbbacbbbbbccccacccbcccaaaccbbbbbcaa  
accabbbccbbbabbbcbbbbbaacbbabbbccccbbbaacaabacccbccaaabaacaaabbcccbabbcaabbbb  
acbbbccabbccccacbccaaabccccbbacbcbaaabccaaacccbccacccbbbaaacccaaabacabbcccaa  
acbcccacbbbaabbbbaaacbbbbcacbbbaacccaaabaacaabbbcbabbcccaaabbbbaaacabbcbbbbab  
bccccbbccaabcacccbccaaabccaaacbabbbccccbacbbbaaacccaaabacaaabbcaabbbcbabbbccc  
acbbaaabbbcbbbbabbccccbbbaacccbccacbaaabbbcccbacbbbabbaabaaacccbacbbbaaccc  
bbbccccacccbccaaabccacccbbbacccaaabacaaabbbcbbbbaccbbccaabcaaabccabcbbbbabbcca  
bbccccbbccaacccbccacccbbacbbbbbccbbbaacaaabaacccbbbbbccccacccbccaaabccaaacccbcc  
cacccbbbaaabbbacaaabacbcccacccbbabbbccaabacccbbbabcbacaaabcccaabbbbaaabcbbbb

aacbbbabbbccbbccaabbbaacabbcccaacbaaabaacbccaccbbaccabaacbbabbcbbaa  
ccbbaccbbbbbccaccbccaabccaccbbbaaaccabbccbbccaabaaacaabccaccbccaabaccbb  
bcccaccbccaabccaaaccbbaacaaabaacccaaaccbbabccaccbccaacbbbaabbcaabbcccc  
aaaccbcccaccbbabbcbbaaaccbcccaccbbbaaacaabaaaccaabbccccbbbabcbbaacc  
aaabbbcbabbccaabbbaaacaabaaaccbcccaccbbbaacccaaacbaaabbbaaccaabbbaaa  
caaabaacccaaabbaccbccacbaaacaabbccaabbbcbabbccaabbbaacaabaaaccbcccac  
cbbbbbccaaabccbbbaacbbaaacaabaaaccaabbccccaaaccbcccaccbbccccaaaccbbbaa  
bcaccbccaabccaabaacaabbbcbbaaccbccaabbcbabbccaabcaaacbcccaccbbacbbcca  
abbbaccbbbbbaccbbabbcbbaaacaabcbbaabbccccaccbbaacccaaabacaaabbcccaa  
bbbccbbbabbbcbbaaccbccaabaaccbcccaccbbccaabcbbaaaccbbccaccbccaabaa  
acaabbbcbabbccabbbaaacccaabaacaaabbcbabcccabaaaccabccaabbcbbbaccbccc  
accbbaaccbaccaabaacaabbbbaabbccbbbabbbcbbaaabbcaabbccccbbaaacbccaabb  
cbbaacaabaaccaabbbaaacaabaaaccbaacccaaaccbbabcccbaacaabbccccbbabbccaa  
abaaacaabbbcccbacaaabaaaccbcccaccbbacbccaaabcbabbccaabbcaabbcbbbabcc  
cbbaaccbccaacccbbabbccaabbbaacaabaacccaaaccbbaaacbbccaccbccaacccbbbc  
caabbbaaacabccaabbbaabbbccabcccb